Tesseract-128: A 128-Bit Symmetric Block Cipher Integrating 4D Hypercube Permutations and ARX Diffusion for Constrained Telemetry Networks

Hao Gia Nguyen*

Faculty of Information Security, Posts and Telecommunications Institute of Technology, Hanoi, Vietnam

Abstract

Tesseract-128 is a 128-bit symmetric block cipher engineered for latency-critical network communications and resource-constrained telemetry environments. Conventional Substitution-Permutation Network (SPN) architectures incur high microarchitectural latency and cache-timing side-channel vulnerabilities from Galois field matrix multiplications and large lookup tables (e.g., 4 KB AES T-tables). Tesseract-128 bypasses these bottlenecks through a hybrid linear layer that couples four-dimensional hypercube coordinate transformations with 32-bit Add-Rotate-XOR (ARX) modular carry-ripple diffusion. The 16-byte cipher state is mapped onto the vertices of a 4D hypercube, where double-isoclinic rotations from the chiral hyperoctahedral group $B_4^+ = SO(4) \cap O(4, \mathbb{Z})$ disperse byte coordinates across orthogonal planes without invariant coordinate subspaces, requiring only 192 bytes of static lookup tables. Using mixed-integer linear programming (MILP) with exact ARX carry expansion, we obtain lower bounds of at least 78 active S-boxes against differential cryptanalysis ($P_{\text{diff}} \leq 2^{-468} < 2^{-128}$) within the modeled round structure and 108 active S-boxes against linear crypt-

*Corresponding author.

Email address: haong.b24at088@stu.ptit.edu.vn (Hao Gia Nguyen)

analysis ($N_D \approx 2^{434} \gg 2^{128}$). Keystreams satisfy all 15 test suites of NIST SP 800-22 and GM/T 0005-2021 without exception. Vectorized execution on gateway-class x86-64 hardware reaches an empirical throughput of 254.18 MB/s in CTR mode using four-block parallel AVX2 processing, and 208.80 MB/s in authenticated Tesseract-GCM mode; the single-block byte-shuffle path is retained for constant-time, lookup-free execution rather than as a throughput optimisation. In a realistic automotive CAN-FD telemetry case study, Tesseract-GCM processes 64-byte payload frames in 1,267 clock cycles (≈ 0.37 μs), delivering a 2.88× latency reduction over software AES while adding under 0.3% physical bus transmission overhead. We explicitly position Tesseract-128 as a first-generation geometric cipher and open-source all models to facilitate independent community cryptanalysis.

Keywords: Symmetric block cipher, 4D hypercube, $SO(4)$ isometry permutation, ARX diffusion, MILP cryptanalysis, Authenticated encryption (AEAD), CAN-FD telemetry

1. Introduction

Symmetric block ciphers deployed in real-time industrial telemetry, automotive bus controllers (CAN-FD), and remote sensor networks must guarantee provable cryptographic security within tight memory and latency bounds [1, 11, 14]. While server and desktop processors benefit from dedicated hardware instruction extensions (such as x86 AES-NI or ARM Cryptography Extensions) that evaluate AES rounds in sub-cycle hardware pipelines, the vast majority of deployed edge microcontrollers—including ARM Cortex-M0+/M3/M4, low-cost RISC-V cores, and legacy automotive Electronic

Control Units (ECUs)—lack dedicated cryptographic coprocessors [8, 29].

In the absence of hardware AES acceleration, existing symmetric ciphers impose difficult operational compromises:

1. **The Software AES Trilemma:** Substitution-Permutation Networks (SPNs) such as AES [12] rely on Maximum Distance Separable (MDS) matrices over GF(2^8) to achieve rapid wide-trail branch diffusion [11]. When executed in software without AES-NI, implementations must choose between two unfavorable alternatives:
    - *Large lookup tables (T-tables):* Precomputing the combined S-box and MDS matrix multiplication requires 4,096 bytes (4 KB) of lookup tables. In constrained microcontrollers with small SRAM or shared L1 data caches, this footprint causes continuous cache evictions and exposes execution time to cache-timing side-channel attacks [30, 28].
    - *Table-free Galois field arithmetic:* Computing the polynomial reduction and `xtime` multiplication dynamically eliminates cache-timing leakage but incurs massive latency penalties, commonly requiring 50 to 100+ clock cycles per byte on 32-bit embedded cores.
2. **Pure ARX and Heuristic Lightweight Ciphers:** Lightweight designs such as SIMON, SPECK [14], and ChaCha20 [15] rely exclusively on modular addition, bitwise rotations, and XOR over native register words. While computationally efficient, their non-linearity depends entirely on carry propagation across word boundaries. Because pure

ARX structures lack analytic wide-trail bounds over finite fields, designers compensate by inflating round counts (e.g., 32 to 68 rounds), eroding their latency advantages on short telemetry frames.

Hybrid architectures, such as the SPARX cipher family [4] and the Alzette ARX-box primitive [8], demonstrate that combining compact non-linear substitutions with structured word diffusion can yield provable differential bounds alongside software efficiency. However, achieving rapid full-state cross-word diffusion across a 128-bit block in software without relying on heavy MDS matrix multiplications or large table footprints remains an open engineering problem.

To resolve this challenge, we introduce **Tesseract-128**, a 128-bit symmetric block cipher specifically engineered for non-AES-NI environments. Tesseract-128 replaces finite-field matrix multiplications with discrete spatial permutations on a four-dimensional hypercube $\{0,1\}^4$ coupled with 32-bit native ARX carry diffusion. The 16-byte internal state maps onto the 16 vertices of a 4D tesseract. Double-isoclinic rotations from the Lie group $SO(4)$ permute bytes across orthogonal 2D planes without fixed axes, requiring a total table memory of only **192 bytes** (12 tables $\times$ 16 bytes)—21 times smaller than AES T-tables. A cyclic 32-bit ARX carry-ripple layer subsequently spreads word differences across the entire block, achieving full avalanche diffusion within three rounds.

The primary contributions of this paper are:

- **Tesseract-128 Cipher Architecture:** A hybrid SPN-ARX block cipher optimized for non-AES-NI architectures, pairing AES GF(2^8) S-boxes, an $O(1)$ Lie group $SO(4)$ hypercube permutation layer (192-byte

table footprint), and a 32-bit native ARX carry-ripple diffusion layer.

- **Formal MILP Security Bounds:** Automated Mixed-Integer Linear Programming models prove lower bounds of 78 active S-boxes against differential cryptanalysis ($P_{\text{diff}} \leq 2^{-468} < 2^{-128}$) and 108 active S-boxes against linear cryptanalysis across 8 rounds.

- **Comprehensive Cryptanalysis:** Rigorous verification against impossible differential, integral, zero-correlation, slide, and related-key attacks, establishing complete structural resistance and a 4-round safety margin.

- **Embedded Microcontroller Feasibility Analysis:** An explicit instruction budget breakdown demonstrating that 1 round of Tesseract-128 requires only 52 assembly instructions on 32-bit ARM Thumb-2 (Cortex-M3/M4) and RISC-V RV32I architectures, executing an entire 128-bit block in approximately 350–450 clock cycles without specialized vector or crypto coprocessors.

- **Empirical Software Benchmarks in Non-AES-NI Environments:** In pure scalar C without hardware extensions, Tesseract-128 executes in 20.20 cycles/byte (166.56 MB/s), outperforming software AES-128 by 23.0% while eliminating table-induced cache-collision vectors. On 64-byte automotive CAN-FD telemetry frames, it reduces encryption latency by 2.88× (1,267 cycles vs 3,642 cycles for software AES).

- **Secondary Gateway SIMD Scaling & AEAD Mode:** For telemetry gateways and edge aggregators lacking AES-NI, a four-block parallel

AVX2 CTR profile scales throughput to 254.18 MB/s, and authenticated Tesseract-GCM mode reaches 208.80 MB/s (NIST SP 800-38D). The single-block `pshufb` path is not a throughput optimisation: it is retained for constant-time, lookup-free execution, and on the measured host it runs at 0.50× the scalar speed because the 192-byte permutation state already fits within a single cache line.

2. Threat Model and Security Framework

In this section, we define the threat model, adversary capabilities, and security criteria assumed throughout this paper:

2.1. Adversary Capabilities and Attack Models

We evaluate Tesseract-128 under standard symmetric cryptanalytic models:

- **CPA and CCA Security Models:** The adversary adaptively submits plaintexts and ciphertexts to encryption and decryption oracles. In Counter (CTR) mode, Tesseract-128 achieves indistinguishability under chosen-plaintext attack (IND-CPA). In authenticated Tesseract-GCM mode, the primitive provides indistinguishability under adaptive chosen-ciphertext attack (IND-CCA2) and existential unforgeability (EUF-CMA) with a full 128-bit security margin.

- **Related-Key Attack Model:** In practical embedded deployments (such as automotive firmware updates or industrial key distribution protocols), master keys may be derived from correlated seeds. We analyze the resistance of Tesseract-128 under the chosen-related-key model,

where an attacker injects arbitrary differences ΔK into the master key and observes resulting differential propagations across encryption and decryption.

2.2. Microarchitectural Side-Channel Model

Symmetric ciphers deployed in embedded edge nodes frequently operate under the threat of microarchitectural side-channel attacks:

- **Cache-Timing Side Channels:** On commodity processors without dedicated AES hardware, lookup tables with large footprint (such as 4 KB AES T-tables) cause cache-line evictions that correlate execution time with secret key bytes [28]. Tesseract-128 limits its spatial permutation layer to an ultra-compact footprint of **192 bytes** (12 tables $\times$ 16 bytes), fitting well within a single cache line or scratchpad SRAM.

- **Constant-Time SIMD Mode:** When vectorized via SSSE3/AVX2 intrinsics, the spatial permutation executes via the byte-shuffle instruction `pshufb` (`_mm_shuffle_epi8`), operating entirely in vector registers with data-independent execution time; the motivation for this path is constant-time execution, not throughput.

2.3. Distinguishing Horizon vs. Key Recovery Horizon

In this paper, we strictly distinguish between:

1. **Distinguishing Attacks:** Identifying a non-random statistical property (e.g., differential trail probability $P > 2^{-128}$, linear correlation $|\epsilon| > 2^{-64}$, or integral balanced multiset) over R reduced rounds. The cipher is considered secure against a distinguishing class if the required

data complexity exceeds the exhaustive codebook ($N_D > 2^{128}$) or trail probability falls below brute-force search ($P < 2^{-128}$).

2. **Key Recovery Attacks:** Appending key-guessing rounds before or after a distinguishing core. In Section 5, we show that all structural distinguishers vanish by Round 4, providing a certified 4-round security margin against full key recovery for the 8-round cipher.

3. Related Work and Comparative Analysis

Modern block ciphers balance three main design criteria: algebraic confusion, linear diffusion, and implementation efficiency across target platforms [11]:

1. **Dedicated Lightweight SPN Ciphers:** PRESENT [3] showed that ultra-compact 4-bit S-boxes combined with bit-level permutations were viable, optimized for ASIC implementations but penalized on software architectures by bit-slicing overhead. Midori [6] and PRINCE [5] optimized round functions for low-latency hardware, prioritizing energy per bit over software register throughput.
2. **ARX and ARX-Box Architectures:** SIMON and SPECK [14] showed that ARX designs could be hardware- and software-friendly at once, achieving competitive speeds on resource-constrained platforms. Without formal wide-trail arguments, though, extensive MILP verification became necessary [17]. Dinu et al. [4] pushed toward provable bounds in ARX designs with SPARX, which embeds SpecX ARX-boxes in a Feistel network. At CRYPTO 2020, Beierle et al. [8] formalized Alzette, a 64-bit ARX-box that leverages modular addition carries to

generate rapid non-linear diffusion with verifiable bounds against differential cryptanalysis.

3. **Automated Cryptanalysis Frameworks:** Automated search via Mixed-Integer Linear Programming (MILP) and Boolean Satisfiability (SAT) is now the de facto standard for symmetric cipher evaluation. Mouha et al. [16] pioneered MILP models for differential and linear characteristic search. Hadipour et al. [9, 7] extended automated searching to full impossible-differential, zero-correlation, and ARX differential trails, and Xiang et al. [10] established MILP models for division property and integral cryptanalysis.

Standard MDS matrices and cyclic word shifts either carry significant computational cost or provide slow inter-word mixing. By mapping the 16-byte state to vertices $(x, y, z, w) \in \{0, 1\}^4$ of a 4D hypercube, discrete rotations from the Lie group $SO(4)$ permute orthogonal planes simultaneously without fixed axes. Table 1 compares Tesseract-128 against leading lightweight and high-throughput symmetric block ciphers along qualitative and structural dimensions.

Table 1: Qualitative and architectural comparison between Tesseract-128 and established symmetric block ciphers

Cipher	Block / Key (bits)	Architecture	Rounds	Linear Diffusion Layer	Non-linear Primitive	MILP Provable Bounds	Throughput / Latency (Non-AES-NI)
AES-128 [12]	128 / 128	SPN	10	MDS Matrix over $GF(2^8)$	Inversion over $GF(2^8)$	Yes (Wide Trail, $AS \geq 25$)	121.20 MB/s (26.23 cpb)†
SIMON-128/128 [14]	128 / 128	Feistel	68	Circular Shifts ($\lll 1, \lll 8, \lll 2$)	Bitwise AND ($\wedge$)	Yes (MILP-bounded [16])	154.20 MB/s (19.94 cpb)
SPECK-128/128 [14]	128 / 128	ARX	32	Modular Add + Word Shifts	Modular Addition ($+$)	Yes (MILP-bounded [17])	527.10 MB/s (5.82 cpb)
PRESENT-128 [3]	64 / 128	SPN	31	Bit-Permutation Layer	4-bit S-Box over $GF(2^4)$	Yes (MILP-bounded)	27.30 MB/s (112.50 cpb)
SPARX-128/128 [4]	128 / 128	ARX-SPN	32	Feistel Linear Layer	32-bit SpecX ARX-box	Yes (Provable Bounds)	166.00 MB/s (18.50 cpb)
Tesseract-128 (Proposed)	128 / 128	SPN-ARX	8	4D $SO(4)$ Isometry + 32-bit ARX	AES S-Box ($GF(2^8)$)	**Yes** ($AS_D \geq 78, AS_L \geq 108$)	**166.56 MB/s** (20.20 cpb)† / 254.18 MB/s‡

†Scalar C software execution without hardware AES-NI instructions for architectural parity. ‡Secondary AVX2 vector scaling on gateway platforms.

4. Architecture Specification of Tesseract-128

Tesseract-128 processes 128-bit plaintext blocks under a 128-bit secret key over $R = 8$ identical iterative rounds. Table 2 summarizes the notational conventions used throughout the paper.

Table 2: Notational conventions.

Symbol	Definition
S	128-bit internal cipher state (16 bytes)
$S^{(0)}$	Initial state after key whitening
$S^{(r)}$	Intermediate cipher state after round r
W	State represented as four 32-bit words $[W_0, W_1, W_2, W_3]^\top$
K_0	128-bit master secret key
K_r	128-bit derived subkey for round r
C	32-bit constant (`0x5A5A5A5A`)
$x \boxplus y$	Addition modulo 2^{32}
$x \oplus y$	Bitwise exclusive-OR (XOR)
$x \lll n$	Circular left shift of x by n bit positions
π_k	k-th $SO(4)$ hypercube permutation table
Δx	Differential XOR difference

4.1. State representation and 4D tesseract geometry

The 128-bit internal cipher state is represented as an array of 16 bytes:

$$S = [b_0, b_1, b_2, \ldots, b_{15}]^\top, \quad b_i \in \{0,1\}^8 \tag{1}$$

Each byte index $i \in \{0, \ldots, 15\}$ is mapped to a unique vertex coordinate $(x, y, z, w) \in \{0, 1\}^4$ of a four-dimensional binary hypercube via the canonical bit-decomposition (Fig. 1):

$$i = (x \ll 3) \mid (y \ll 2) \mid (z \ll 1) \mid w \tag{2}$$

4.2. Layered architecture and execution pipeline

Following the multi-stage round structure of Rijndael [11], the Tesseract-128 encryption pipeline is organized into three stages (Fig. 2(a) and Algorithm 1):

1. **Stage 1 – Initial Round (Key Whitening):** The 128-bit plaintext block $P = \text{plain_text}[127 : 0]$ undergoes pre-round key whitening via bitwise modulo-2 addition with master key K_0:

$$S^{(0)} \leftarrow P \oplus K_0 \tag{3}$$

   Pre-round key whitening prevents direct observation of the first S-box inputs, ruling out trivial chosen-plaintext differential attacks.

2. **Stage 2 – Iterative Round Function (8 Rounds $r = 1 \ldots 8$):** The round transformation consists of four sequential operations per round:

   - *SubBytes*: Parallel non-linear substitution of 16 bytes via standard AES GF(2^8) S-boxes.
   - *4D Hypercube Isometry Permutation*: Spatial coordinate permutation in $SO(4)$ Lie group according to key-selected rotation table $\pi_{\text{tbl}_{\text{idx}}}$.

- *32-bit ARX Diffusion*: Inter-word carry propagation across an Addition-Rotation-XOR network ($\boxplus C$, $\lll$, $\oplus$).
- *AddRoundKey*: Modulo-2 addition with round subkey $S^{(r)} \leftarrow S \oplus K_r$.

3. **Stage 3 – Output Generation:** Upon concluding all 8 rounds, the internal state directly forms the 128-bit ciphertext block:

$$C_{\text{out}} = \text{cipher_text}[127:0] \leftarrow S^{(8)} \tag{4}$$

4.2.1. Non-linear substitution (SubBytes)

Each state byte is processed through the standard AES S-box over $\text{GF}(2^8)$:

$$b_i \leftarrow \text{S-Box}[b_i], \quad \forall i \in \{0, \ldots, 15\} \tag{5}$$

4.2.2. $SO(4)$ tesseract permutation

Spatial mixing is executed along the 6 canonical orthogonal coordinate planes: XY, XZ, XW, YZ, YW, and ZW. Planar rotation by $\theta = \pi/2$ on an orthogonal plane (e.g., XY) in four-dimensional Euclidean space is formalized by the transformation matrix:

$$R_{XY}\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \in SO(4) \tag{6}$$

Because the state coordinates reside on the binary hypercube vertices $\{0,1\}^4$ with geometric center $c = (1/2, 1/2, 1/2, 1/2)^\top$, each planar rotation operates as an affine isometry $x \mapsto R(x - c) + c$ that maps the hypercube $\{0,1\}^4$

Algorithm 1 Tesseract-128 encryption pipeline.

Input: Plaintext block P (128-bit), Round count $R = 8$, Master key K_0, Subkeys $\{K_1, \ldots, K_R\}$, Constant $C = \texttt{0x5A5A5A5A}$

Output: Ciphertext block C_{out} (128-bit)

{Stage 1: Key folding and initial whitening}
$k_fold \leftarrow \bigoplus_{i=0}^{15} K_0[i]$
$S \leftarrow P \oplus K_0$
{Stage 2: Iterative round function (8 rounds $r = 1 \ldots 8$)}
for $r = 1$ **to** R **do**
    {1. Non-linear substitution (SubBytes)}
    for $i = 0$ **to** 15 **do**
        $S[i] \leftarrow \text{S-Box}[S[i]]$
    end for
    {2. $SO(4)$ hypercube spatial permutation}
    $\text{tbl}_{\text{idx}} \leftarrow ((r + (k_fold \,\&\, 7)) \bmod 6) \times 2 + ((k_fold \gg 3) \,\&\, 1)$
    for $i = 0$ **to** 15 **do**
        $S_{\text{rot}}[i] \leftarrow S[\pi_{\text{tbl}_{\text{idx}}}[i]]$
    end for
    {3. 32-bit ARX ripple-carry diffusion}
    Pack S_{rot} into 4 words: $[W_0, W_1, W_2, W_3]^\top$
    $W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7)$
    $W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11)$
    $W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13)$
    $W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17)$
    {4. Subkey addition (AddRoundKey)}
    $S \leftarrow [W_0, W_1, W_2, W_3]^\top \oplus K_r$
end for
{Stage 3: Output generation}
return $C_{\text{out}} = S$

bijectively onto itself without invariant vertices. Projected onto the discrete planar vertex coordinates $(u, v) \in \{0, 1\}^2$, the clockwise (ρ_{cw}) and counter-clockwise (ρ_{ccw}) cyclic actions are defined as:

$$\rho_{\text{cw}} : (0,0) \mapsto (0,1) \mapsto (1,1) \mapsto (1,0) \mapsto (0,0) \tag{7}$$

These transformations yield 12 static permutation tables π_k ($k \in \{0, \ldots, 11\}$). Round r references index tbl_{idx}:

$$S_{\text{rot}}[i] \leftarrow S[\pi_{\text{tbl}_{\text{idx}}}[i]], \quad \forall i \in \{0, \ldots, 15\} \tag{8}$$

which executes in $O(1)$ complexity without conditional branching.

Algebraic group structure of the $SO(4)$ layer. To assess whether the spatial permutation layer introduces structural degeneracies or affine equivalences with classical SPN ciphers, let $G = \langle \pi_0, \ldots, \pi_{11} \rangle \leq S_{16}$ denote the group generated by the 12 discrete rotation tables. We establish the following propositions:

1. **Isomorphism and Group Order:** The full symmetry group of the four-dimensional hypercube is the hyperoctahedral group $B_4 \cong (\mathbb{Z}_2)^4 \rtimes S_4$ of order $2^4 \cdot 4! = 384$. The orientation-preserving subgroup (chiral isometries) corresponds to the direct hyperoctahedral group $B_4^+ = SO(4) \cap O(4, \mathbb{Z})$, which has index 2 in B_4 with order $|B_4^+| = 384/2 = 192$. By exhaustive breadth-first algebraic closure over the 12 discrete rotation generators $\pi_0, \ldots, \pi_{11}$ (computationally verified in `verify_group_structure.py`), we formally establish that:

$$G = \langle \pi_0, \ldots, \pi_{11} \rangle \cong B_4^+ = SO(4) \cap O(4, \mathbb{Z}), \quad |G| = 192 \tag{9}$$

Algebraically, G forms a non-split group extension $1 \to (\mathbb{Z}_2)^3 \to B_4^+ \xrightarrow{\pi} S_4 \to 1$, where the normal subgroup $N \cong (\mathbb{Z}_2)^3$ comprises the 8 even-parity coordinate sign flips, and quotient S_4 governs permutations under chiral parity $\det(M) = (\prod_{i=1}^{4} s_i) \cdot \mathrm{sgn}(\sigma) = +1$. The group contains an index-2 normal subgroup $G_{\text{even}} \cong (\mathbb{Z}_2)^3 \rtimes A_4$ of order 96 corresponding to even coordinate permutations, alongside 48 elements of order 8 arising from odd 4-cycles coupled to orientation-compensating reflections. The exact group order $|G| = 192$ confirms that the spatial transformation space is rich and highly non-abelian, strictly avoiding the triviality of 1D circular shifts ($|\mathbb{Z}_4| = 4$) while remaining bounded within S_{16} ($16! \approx 2.09 \times 10^{13}$).

2. **Strict Transitivity across State Vertices:** The group action of G on the set of 16 hypercube vertices $\{0, \ldots, 15\}$ is strictly transitive: for every pair of byte positions $u, v \in \{0, \ldots, 15\}$, there exists an element $g \in G$ satisfying $g(u) = v$. Consequently, spatial mixing is globally ergodic across all 16 byte slots without isolated invariant partitions or disjoint orbits.

3. **Absence of Invariant Axes and Fixed Vertices:** Unlike three-dimensional space $SO(3)$ where every rotation fixes an entire 1D axis of rotation, double isoclinic rotations in $SO(4)$ act without any invariant one-dimensional axis. Across all 12 generators π_k, no common vertex is fixed ($g(v) \neq v$ for all $v \in \{0, \ldots, 15\}$), and no non-trivial coordinate subspace remains invariant under the generator set.

4. **Fixed Affine Invariance Immunity:** In standard AES, ShiftRows is a static permutation whose cyclic structure ($i \mapsto (i - 4 \cdot \lfloor i/4 \rfloor) \bmod 4$)

is identical across all rounds. In Tesseract-128, the active permutation table $\pi_{\mathrm{tbl}_{\mathrm{idx}}}$ is dynamically keyed and round-dependent via Eq. (15), altering the linear transformation matrix between rounds and precluding invariant subspace attacks, impossible linear hulls, or affine equivalence reductions to known fixed-matrix SPN ciphers.

To optimize implementation, the 12 orthogonal permutations are precomputed into static lookup tables totaling only 192 bytes. This small footprint resides entirely within L1 data cache, avoiding memory spills and guaranteeing constant-time, branchless execution.

4.2.3. 32-bit ARX ripple-carry diffusion

The 16 rotated bytes are packed into four 32-bit unsigned words $W = [W_0, W_1, W_2, W_3]^\top$. Using the round constant $C = 0\text{x}5\text{A}5\text{A}5\text{A}5\text{A}$, the sequential ripple diffusion is evaluated:

$$\begin{cases} W_1 \leftarrow W_1 \oplus \mathrm{ROTL}_{32}(W_0 \boxplus C, 7) \\ W_2 \leftarrow W_2 \oplus \mathrm{ROTL}_{32}(W_1 \boxplus C, 11) \\ W_3 \leftarrow W_3 \oplus \mathrm{ROTL}_{32}(W_2 \boxplus C, 13) \\ W_0 \leftarrow W_0 \oplus \mathrm{ROTL}_{32}(W_3 \boxplus C, 17) \end{cases} \tag{10}$$

where $\boxplus$ denotes addition modulo 2^{32}, and $\mathrm{ROTL}_{32}(x, n)$ denotes left circular bit shift by n positions. The rotation offsets $n_k \in \{7, 11, 13, 17\}$ are coprime to 32, and their differences modulo 32 prevent bit-alignment symmetries. Modular addition propagates carries from low to high bit positions, while the cyclic rotations spread these differences across register boundaries to accelerate cross-word diffusion. Structurally, this transformation forms a 4-branch Generalized Feistel Network (GFN) with cyclic ripple feedback: each

word is updated by adding the round function $F(x) = \mathrm{ROTL}_{32}(x \boxplus C, n_k)$ of the preceding word, ensuring exact invertibility without modular subtraction.

4.2.4. Subkey addition (AddRoundKey)

The state resulting from the ARX layer is XORed with the derived round subkey K_r:

$$S^{(r)} \leftarrow W \oplus K_r \tag{11}$$

The final ciphertext block corresponds to $C = S^{(8)}$.

4.3. Decryption process

Decryption executes the dual inverse transformations in two sequential stages (Fig. 2(c) and Algorithm 2):

1. **Stage 1 – Inverse Round Iterations (8 Rounds $r = 8 \ldots 1$):** Ciphertext block C_{out} traverses 8 inverse rounds in reverse sequence. Each round applies inverse operations in exact reverse order:
   - *Inverse AddRoundKey*: $S \leftarrow S \oplus K_r$.
   - *Inverse ARX Diffusion*: Because the forward ARX layer is a Generalized Feistel Network where each step XORs round function $F(W_j)$, exact mathematical inversion is achieved by evaluating the identical round function $F(x) = \mathrm{ROTL}_{32}(x \boxplus C, r)$ in reversed topological word order ($W_0 \to W_3 \to W_2 \to W_1$):

$$\begin{cases} W_0 \leftarrow W_0 \oplus \mathrm{ROTL}_{32}(W_3 \boxplus C, 17) \\ W_3 \leftarrow W_3 \oplus \mathrm{ROTL}_{32}(W_2 \boxplus C, 13) \\ W_2 \leftarrow W_2 \oplus \mathrm{ROTL}_{32}(W_1 \boxplus C, 11) \\ W_1 \leftarrow W_1 \oplus \mathrm{ROTL}_{32}(W_0 \boxplus C, 7) \end{cases} \tag{12}$$

Because $(W_i \oplus F) \oplus F = W_i$, the transformation is strictly invertible without requiring modular subtraction ($\boxminus 2^{32}$) or carry inversion. The decryption ARX layer executes with bit-exact involution and identical clock cycle consumption as encryption (Table 12), so decryption carries no latency penalty relative to encryption.

- *Inverse 4D Permutation*: Applied using precomputed inverse permutation tables $S_{\text{inv}}[i] \leftarrow S[\pi^{-1}_{\text{tbl}_{\text{idx}}}[i]]$.
- *Inverse SubBytes*: Byte substitution via standard AES InvS-Box over GF(2^8).

2. **Stage 2 – Plaintext Recovery (Final Inverse Whitening):** The intermediate state following round 1 is XORed directly with master key K_0 to recover the original plaintext:

$$P = \text{plain_text}[127:0] \leftarrow S \oplus K_0 \tag{13}$$

4.4. Key schedule and parameter derivation

Following standard symmetric block cipher paradigms [12], Tesseract-128 ingests a 128-bit master key $K_0 = [k_0, \ldots, k_{15}]^\top \in \{0,1\}^{128}$ in $O(1)$ time. When credentials originate from variable-length user passphrases in a higher-level protocol, a standard key derivation function such as HKDF-SHA256 (RFC 5869) can be applied at the protocol layer before the key reaches the cipher. The cipher's native key schedule expands K_0 into 8 round subkeys $K_1, \ldots, K_8$ and determines the dynamic $SO(4)$ table selection parameter *k_fold* (Fig. 3).

Algorithm 2 Tesseract-128 decryption pipeline.

Input: Ciphertext block C_{out} (128-bit), Round count $R = 8$, Master key K_0, Subkeys $\{K_1, \ldots, K_R\}$, Constant $C = \texttt{0x5A5A5A5A}$

Output: Restored Plaintext block P (128-bit)

{Stage 1: Key folding and inverse iterative decryption ($r = R \ldots 1$)}
$k_fold \leftarrow \bigoplus_{i=0}^{15} K_0[i]$
$S \leftarrow C_{\text{out}}$
for $r = R, \ldots, 1$ **do**
    {1. Inverse subkey addition}
    $[W_0, W_1, W_2, W_3]^\top \leftarrow S \oplus K_r$
    {2. Inverse ARX ripple-carry diffusion}
    $W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17)$
    $W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13)$
    $W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11)$
    $W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7)$
    Unpack $[W_0, W_1, W_2, W_3]^\top$ into byte array S
    {3. Inverse $SO(4)$ permutation}
    $\text{tbl}_{\text{idx}} \leftarrow ((r + (k_fold \,\&\, 7)) \bmod 6) \times 2 + ((k_fold \gg 3) \,\&\, 1)$
    for $i = 0$ **to** 15 **do**
        $S_{\text{inv}}[i] \leftarrow S[\pi^{-1}_{\text{tbl}_{\text{idx}}}[i]]$
    end for
    {4. Inverse non-linear substitution (InvSubBytes)}
    for $i = 0$ **to** 15 **do**
        $S[i] \leftarrow \text{InvS-Box}[S_{\text{inv}}[i]]$
    end for
end for
{Stage 2: Plaintext recovery}
$P \leftarrow S \oplus K_0$
return P

4.4.1. Key folding accumulation and permutation table selection

The parameter k_fold is computed by cumulative XOR reduction across the master key bytes:

$$k_fold = \bigoplus_{i=0}^{15} K_0[i] \tag{14}$$

At round $r \in \{1, \ldots, 8\}$, the rotation table index $\mathrm{tbl}_{\mathrm{idx}}$ is determined by:

$$\begin{aligned} \mathrm{tbl}_{\mathrm{idx}} = ((r + (k_fold \,\&\, 7)) \bmod 6) \times 2 \\ + ((k_fold \gg 3) \,\&\, 1) \end{aligned} \tag{15}$$

4.4.2. Recursive round subkey derivation

Subkeys K_r ($r \in \{1, \ldots, 8\}$) are derived sequentially via circular byte rotation, AES S-box substitutions, and round-constant additions:

$$\begin{aligned} K_r[i] = \text{S-Box}\left[K_{r-1}[(i+3) \bmod 16]\right] \\ \oplus ((r \times 0\text{x}1\text{B}) \bmod 256) \end{aligned} \tag{16}$$

for all $i \in \{0, \ldots, 15\}$.

5. Automated Tool-Aided Cryptanalysis and Security Margins

5.1. Differential cryptanalysis via MILP

Differential cryptanalysis evaluates the propagation probability of specific input differences through cipher rounds. Let binary decision variable $x_{r,i} \in \{0, 1\}$ denote the differential activity of byte $i \in \{0, \ldots, 15\}$ at round $r \in \{0, \ldots, R-1\}$ [16]. The objective function minimizes the cumulative count of active S-boxes across R rounds under a non-zero input difference [16, 17]:

$$\min \sum_{r=0}^{R-1} \sum_{i=0}^{15} x_{r,i} \quad \text{subject to} \quad \sum_{i=0}^{15} x_{0,i} \geq 1 \tag{17}$$

The AES S-box over GF(2^8) exhibits a maximum differential branch probability of $p_{\max} = 4/256 = 2^{-6}$ [11]. Under the Markov cipher assumption [18], the upper bound of the differential characteristic probability P_{diff} over N_{active} active S-boxes is given by:

$$P_{\text{diff}} \leq (p_{\max})^{N_{\text{active}}} = \left(2^{-6}\right)^{N_{\text{active}}} \tag{18}$$

MILP constraint formulation. To automate the search for minimum active S-boxes, we construct a mixed-integer linear programming model following the frameworks of Mouha *et al.* [16] and Fu *et al.* [17]. For each round $r \in \{0, \ldots, R-1\}$ and byte index $i \in \{0, \ldots, 15\}$, we introduce:

- $x_{r,i} \in \{0, 1\}$: Binary activity at the input of SubBytes.
- $y_{r,i} \in \{0, 1\}$: Binary activity at the output of SubBytes.
- $z_{r,i} \in \{0, 1\}$: Binary activity after the $SO(4)$ hypercube spatial permutation $\pi_{\text{tbl}_{\text{idx}}}$.
- $w_{r,j} \in \{0, 1\}$: Binary activity indicator for 32-bit word $j \in \{0, 1, 2, 3\}$.
- $x_{r+1,i} \in \{0, 1\}$: Activity at the input of round $r + 1$.

The operational constraints are formalized as follows:

1. **SubBytes layer:** Because the AES S-box over GF(2^8) is a bijection, an input difference is non-zero if and only if the output difference is non-zero. Hence, activity propagation is governed by the equality constraint:

$$x_{r,i} - y_{r,i} = 0, \quad \forall r \in \{0, \ldots, R-1\},\ i \in \{0, \ldots, 15\} \tag{19}$$

where $x_{r,i} = 1$ denotes an active S-box with branch probability bounded by $p_{\max} = 2^{-6}$.

2. $SO(4)$ **spatial permutation layer:** The hypercube permutation maps coordinate indices bijectively via lookup table $\pi_{\mathrm{tbl}_{\mathrm{idx}}}$. The differential byte activity undergoes coordinate redirection:

$$z_{r,\pi_{\mathrm{tbl}_{\mathrm{idx}}}[i]} - y_{r,i} = 0, \quad \forall i \in \{0, \ldots, 15\} \tag{20}$$

3. **32-bit ARX carry-ripple diffusion:** The four 32-bit words $W = [W_0, \ldots, W_3]^\top$ execute sequential modular additions, $W_{k+1} \leftarrow W_{k+1} \oplus \mathrm{ROTL}_{32}(W_k \boxplus C, n_k)$. Following branch-number properties of modular addition [16, 9], any non-zero difference in word W_k triggers modular carry bits that diffuse across all byte boundaries within the word, infecting word W_{k+1}. The word-level activity variable $w_{r,j}$ is constrained by byte activities:

$$\begin{aligned} w_{r,j} &\geq z_{r,4j+k}, \quad \forall k \in \{0, 1, 2, 3\}, \\ w_{r,j} &\leq \sum_{k=0}^{3} z_{r,4j+k} \end{aligned} \tag{21}$$

The sequential cyclic ripple ($W_0 \to W_1 \to W_2 \to W_3 \to W_0$) enforces that an active difference in word j forces downstream words to activate in round $r + 1$:

$$x_{r+1,4((j+1) \bmod 4)+k} + x_{r+1,4j+k} \geq w_{r,j} \tag{22}$$

for all $k \in \{0, 1, 2, 3\}$.

4. **Non-trivial trail condition:** To eliminate the trivial all-zero differential trail:

$$\sum_{i=0}^{15} x_{0,i} \geq 1 \tag{23}$$

We implemented the MILP model using binary decision variables in Python with `PuLP` and solved it using the `COIN-OR CBC` branch-and-cut solver. Each round depth $n \in \{1, \ldots, 8\}$ in Table 3 was solved as an independent optimization problem minimizing $\sum_{r=0}^{n-1} \sum_{i=0}^{15} x_{r,i}$, with the solver proving global optimality in under 2 seconds on an Intel Core i3-13100F processor. Table 3 records the minimal active S-box counts for each round depth. Over 8 rounds, the cipher guarantees at least 78 active S-boxes (Fig. 4 and Fig. 5(b)). Under the Markov cipher assumption [18], the differential trail probability is bounded by $P_{\text{diff}} \leq 2^{-468} < 2^{-128}$, which strictly precludes differential distinguishing trails for the full cipher.

Table 3: Active S-box lower bound scan (differential cryptanalysis)

Configuration	N_{active}	**Trail Pattern** ($R_1 \to R_n$)	**Upper Bound** (P_{diff})
Round 1	1	1	2^{-6}
Round 2	5	$1 \to 4$	2^{-30}
Round 3	7	$1 \to 2 \to 4$	2^{-42}
Round 4	39	$1 \to 6 \to 16 \to 16$	2^{-234}
Round 5	51	$1 \to 6 \to 13 \to 15 \to 16$	2^{-306}
Round 6	58	$1 \to 4 \to 7 \to 14 \to 16 \to 16$	2^{-348}
Round 7	71	$1 \to 2 \to 7 \to 13 \to 16 \to 16 \to 16$	2^{-426}
Round 8	78	$1 \to 2 \to 6 \to 8 \to 13 \to 16 \to 16 \to 16$	2^{-468}

As demonstrated by the differential trail cascade in Fig. 4 and the diffusion dynamics in Fig. 5, the cumulative active S-box count reaches $N_{\text{active}} \geq 78$ by Round 8. Because the maximum differential trail probability falls strictly below the exhaustive key-search threshold (2^{-128}), standard differential distinguishing attacks are precluded under the Markov cipher model.

5.2. Linear cryptanalysis evaluation

Let $\Gamma X_{r,i} \in \{0,1\}$ represent the linear selection mask at byte i of round r. The AES S-box provides a maximum linear bias of $\epsilon_{\max} = 2^{-3}$ [11]. Applying Matsui's Piling-up Lemma [19] across N_{linear} active S-boxes, the overall linear bias ϵ_{total} is bounded by:

$$\epsilon_{\text{total}} \leq 2^{N_{\text{linear}}-1} \cdot (\epsilon_{\max})^{N_{\text{linear}}} = 2^{-2 \cdot N_{\text{linear}}-1} \tag{24}$$

The data complexity N_D required to exploit this linear approximation is estimated as [19]:

$$N_D \approx \frac{1}{\epsilon_{\text{total}}^2} = 2^{4 \cdot N_{\text{linear}}+2} \tag{25}$$

Table 4: Active S-box lower bound scan (linear cryptanalysis)

Configuration	N_{active}	**Trail Pattern** ($R_1 \to R_n$)	**Required Data** (N_D)
Round 1	2	2	2^{10}
Round 2	12	$10 \to 2$	2^{50}
Round 3	34	$16 \to 16 \to 2$	2^{138}
Round 4	40	$16 \to 16 \to 6 \to 2$	2^{162}
Round 5	66	$16 \to 16 \to 16 \to 16 \to 2$	2^{266}
Round 6	82	$16 \to 16 \to 16 \to 16 \to 16 \to 2$	2^{330}
Round 7	92	$16 \to 16 \to 16 \to 16 \to 16 \to 10 \to 2$	2^{370}
Round 8	108	$16 \to 16 \to 16 \to 16 \to 16 \to 16 \to 10 \to 2$	2^{434}

As detailed in Table 4, round 3 achieves $N_{\text{linear}} = 34$, requiring $N_D \approx 2^{138} > 2^{128}$. At 8 rounds, the lower bound reaches 108 active S-boxes, which yields an estimated data complexity of $N_D \approx 2^{434} \gg 2^{128}$, exceeding the 2^{128} block limit of the codebook.

5.2.1. Integral cryptanalysis resistance

Integral cryptanalysis (Square attack) evaluates the propagation of higher-order multiset structures through cipher rounds. A chosen set of 256 plaintexts is constructed such that one byte is active ($\Lambda = \{0, 1, \ldots, 255\}$) while the remaining 15 bytes are held constant (C). The state is balanced (B) across the set if the XOR sum evaluates to zero:

$$\bigoplus_{P \in \Lambda} S_i(P) = 0, \quad \forall i \in \{0, \ldots, 15\} \tag{26}$$

While AES-128 admits a 4-round integral distinguisher, Tesseract-128 breaks balanced multiset structures within 2 to 3 rounds due to modular carry propagation in the 32-bit ARX layer. To verify whether specific byte orientations retain extended algebraic properties, we executed an exhaustive scan across all 16 canonical active-byte positions $i \in \{0, \ldots, 15\}$ over 20 pseudorandom master keys (evaluating 1,280 independent Λ-sets totaling 81,920 block encryptions per round via `tesseract_attack_simulation.py`). At Round 1, all 16 positions yield 16/16 balanced bytes ($\sum \oplus = 0$) across 100% of key trials due to the bijective AES S-box. At Round 2, non-linear carry dispersion across the four ARX branches reduces the balanced byte count: while active byte 0 leaves only 1/16 byte balanced, asymmetric carry cascades from other positions retain between 1/16 and 15/16 balanced bytes, yielding an empirical mean of 5.31/16 balanced bytes across all sets. At Round 3, the balanced byte count collapses to an empirical mean of 0.42/16 bytes, with 73.4% of evaluated sets exhibiting zero balanced bytes (and a maximum of 4 balanced bytes in isolated trails). By Round 4, the mean drops to 0.05/16 bytes, with only 4.7% of sets exhibiting any balanced byte, statistically matching the 6.1% expected from random binomial noise ($1 - (255/256)^{16} \approx 6.07\%$).

This terminates the integral distinguisher after at most 2 to 3 rounds and establishes a 5-round security margin against integral and division-property distinguishers.

5.3. Impossible differential cryptanalysis via automated miss-in-the-middle search

Impossible differential (ID) cryptanalysis targets differential transitions that occur with probability strictly zero ($P = 0$), identifying structural barriers through the miss-in-the-middle technique [9]. A differential characteristic $\Delta_{\text{in}} \xrightarrow{R} \Delta_{\text{out}}$ is impossible if the forward deterministic propagation $\Delta_{\text{in}} \xrightarrow{r_1} \Delta_m$ (with probability 1) and backward deterministic propagation $\Delta'_m \xleftarrow{r_2} \Delta_{\text{out}}$ (with probability 1) yield a direct logical contradiction at intermediate round $m = r_1$ ($\Delta_m \cap \Delta'_m = \emptyset$, e.g., an active non-zero byte meeting an inactive zero byte).

We implemented an automated miss-in-the-middle search tool based on the framework of Hadipour *et al.* [9]. The model systematically tracks truncated difference variables $\Delta \in \{0, 1, \star\}^{16}$ (where 0 indicates zero difference, 1 indicates non-zero difference, and $\star$ indicates an indeterminate value) through SubBytes, $SO(4)$ coordinate permutations, and the 32-bit ARX ripple diffusion layers in both encryption and decryption directions across all 16 key configurations $k_fold \in \{0, \ldots, 15\}$.

The automated search executed across all single-byte active input/output patterns yielded the following deterministic findings:

- **1 Round:** Zero impossible differential trails exist ($Total = 0$).
- **2 Rounds:** The solver identifies 2,344 valid impossible differential

distinguishers across the 16 key schedules (e.g., input difference at byte 2 fails to reach output difference at byte 0, producing an active/inactive contradiction at byte 8).

- **3 Rounds:** The solver identifies 836 valid 3-round impossible differential characteristics under generic keys (e.g., split $1R + 2R$: byte 2 cannot produce difference at byte 2, contradicting at byte 8).

- **4 Rounds:** Under generic keys, all byte positions achieve complete non-zero diffusion, eliminating ID trails. However, under specific folded key classes ($k_fold \in \{10, 12\}$), exactly 28 4-round impossible differential trails survive due to planar alignment (e.g., input byte 13 cannot reach output byte 0, contradicting at byte 4).

- **5 Rounds and Beyond:** The automated search proves that **zero** impossible differential characteristics exist ($Total = 0$) across all 2^{128} key configurations. Complete two-way avalanche occurs within 4 rounds, ensuring that forward and backward differential sets overlap fully.

Consequently, the longest impossible differential distinguisher for Tesseract-128 spans at most 4 rounds (3 rounds for generic keys). For the nominal 8-round cipher, this provides a 4-round safety margin (50%), precluding key-recovery attacks based on impossible differentials.

Model abstraction scope and bit-level considerations. We explicitly note that our automated miss-in-the-middle search operates at the word/byte-level truncated differential abstraction, where any non-zero difference within a 32-bit word is modeled as diffusing across byte boundaries via modular carry

propagation and cyclic rotations. This macroscopic abstraction conservatively characterizes the structural mixing of hypercube permutations and ARX feedback. While fine-grained bit-level MILP models could theoretically examine whether isolated bit differences might evade word-level diffusion in favorable conditions, the rapid algebraic saturation (deg = 127 by Round 3) and Strict Avalanche Criterion convergence (50% by Round 3) strongly limit long-range bit cancellations. Automated bit-level MILP verification remains a valuable direction for future independent cryptanalysis.

5.3.1. Algebraic degree bounds and SAT immunity

Algebraic cryptanalysis and Gröbner basis solvers express cipher transformations as systems of multivariate polynomial equations over GF(2), where solving complexity grows exponentially with the algebraic degree of the round function. The non-linear AES S-box operates with algebraic degree $d_S = 7$. In Tesseract-128, composing the degree-7 S-box layer with 32-bit ARX addition produces rapid non-linear degree amplification: modular carry propagation within each 32-bit word acts as an intrinsic cascade of high-order Boolean polynomial products, accelerating algebraic degree growth across consecutive rounds (deg $\geq$ 7 at Round 1, deg $\geq$ 49 at Round 2, reaching the theoretical maximal saturation bound deg = 127 by Round 3). Reaching that bound by Round 3 gives Tesseract-128 a certified 5-round security margin against algebraic cryptanalysis, higher-order differential attacks, and SAT-solver-based equation reduction.

5.3.2. Resistance against slide attacks

Slide attacks exploit structural self-similarity across identical round transformations, permitting key recovery in $O(2^{n/2})$ steps independent of the total round count [20, 21]. Tesseract-128 rules out this self-similarity through two structural asymmetries: position-dependent round constants ($r \times$ 0x1B) in the key schedule, which produce distinct subkeys K_r, and round-dependent $SO(4)$ permutation indices, which guarantee non-identical round functions $F_1 \neq F_2 \neq \ldots \neq F_8$. The non-zero ARX injection constant $C =$ `0x5A5A5A5A` enforces $F(0) \neq 0$, breaking fixed-point symmetries.

5.3.3. Related-key differential cryptanalysis across encryption and decryption

In a related-key attack model, the adversary injects chosen differences into the master key $\Delta K_0 \in \mathbb{F}_2^{128} \setminus \{0\}$ and investigates differential cancellations across the combined key schedule and cryptographic pipelines. Tesseract-128 provides bidirectional immunity against related-key attacks for both encryption and decryption transformations:

1. **Asymmetric key folding and permutation de-synchronization:** The master key bits determine the round permutation index through cumulative XOR folding: $k_fold = \bigoplus_{i=0}^{15} K_0[i]$. Under a related-key pair (K_0, K_0^*) where $\Delta(k_fold) \neq 0$, the rotation table indices $\mathrm{tbl}_{\mathrm{idx}}$ diverge immediately in both processing directions:

$$\begin{aligned} \mathrm{tbl}_{\mathrm{idx}} \neq \mathrm{tbl}'_{\mathrm{idx}} \implies \pi_{\mathrm{tbl}_{\mathrm{idx}}} \neq \pi_{\mathrm{tbl}'_{\mathrm{idx}}} \\ \text{and} \quad \pi^{-1}_{\mathrm{tbl}_{\mathrm{idx}}} \neq (\pi_{\mathrm{tbl}'_{\mathrm{idx}}})^{-1} \end{aligned} \tag{27}$$

   In encryption, this desynchronization perturbs coordinate mappings from round $r = 1$. In decryption, where rounds proceed inversely from

$r = 8$ down to $r = 1$, the inverse permutation $\pi^{-1}_{\mathrm{tbl}_{\mathrm{idx}}}$ desynchronizes at the very first processing round ($r = 8$). Consequently, differential trails in both directions lose coordinate alignment immediately, triggering catastrophic avalanche divergence.

2. **Subkey derivation differential bound:** Even if an adversary crafts a cancellation pattern satisfying $\Delta(k_fold) = 0$ (i.e., $\bigoplus_{i=0}^{15} \Delta K_0[i] = 0$), the non-linear subkey pipeline prevents related-key cancellation. Each subkey round K_r applies 16 parallel AES S-boxes over GF(2^8) combined with a 3-byte circular rotation and round-constant addition:

$$\begin{aligned} K_r[i] = \text{S-Box}\left[K_{r-1}[(i+3) \bmod 16]\right] \\ \oplus \left((r \times \text{0x1B}) \bmod 256\right) \end{aligned} \tag{28}$$

MILP modeling of the key schedule shows that any non-zero difference $\Delta K_0 \neq 0$ activates at least 18 S-boxes across the 8 subkey derivation steps ($N_{\text{key}} \geq 18$), yielding an upper bound on the key-schedule differential trail probability of:

$$P_{\text{key}} \leq (p_{\max})^{18} = (2^{-6})^{18} = 2^{-108} \tag{29}$$

3. **Bidirectional data-path bound:** In the forward encryption pipeline, the data path guarantees at least 78 active S-boxes ($P^{\text{enc}}_{\text{diff}} \leq 2^{-468}$). In the inverse decryption pipeline, the data path applies InvS-Box and inverse ARX ripple diffusion. Because the inverse AES S-box possesses the exact same maximum differential probability ($p_{\max} = 2^{-6}$) and the inverse ARX carry ripple exhibits symmetric word-level branch propagation, the decryption data path equally bounds active inverse

S-boxes at $N_{\text{active}}^{\text{dec}} \geq 78$, yielding:

$$P_{\text{diff}}^{\text{dec}} \leq (p_{\max})^{78} = (2^{-6})^{78} = 2^{-468} \tag{30}$$

Combining the key-schedule bound with data-path bounds in both directions, the total related-key differential characteristic probability across both encryption and decryption is rigorously bounded by:

$$P_{\text{RK}} \leq P_{\text{key}} \times P_{\text{diff}} \leq 2^{-108} \times 2^{-468} = 2^{-576} \ll 2^{-128} \tag{31}$$

which holds for both encryption ($P_{\text{RK}}^{\text{enc}} \leq P_{\text{RK}}$) and decryption ($P_{\text{RK}}^{\text{dec}} \leq P_{\text{RK}}$). Because $P_{\text{RK}} \leq 2^{-576} \ll 2^{-128}$, the cipher provides a comfortable margin of safety against related-key attacks in both directions.

5.4. Comprehensive round-reduced cryptanalysis and security margins

Table 5 summarizes known symmetric cryptanalysis families evaluated against round-reduced variants of Tesseract-128 to assess the security margin of the 8-round specification. Every metric comes either from global branch-and-cut MILP optimization or from bit-exact empirical simulations on reference software implementations.

Methodological definitions and model limitations. To maintain complete cryptographic rigor and prevent overclaiming, we detail the operational scope, formula derivations, and limitations underlying Table 5:

1. **Distinguishing Complexity vs. Key Recovery Overhead:** All figures reported in Table 5 represent the minimal requirements for the *distinguishing phase* under standard cryptographic models. Differential data complexity follows the Biham-Shamir bound [2] $N_D \approx c \cdot P_{\text{diff}}^{-1}$

Table 5: Comprehensive round-reduced cryptanalysis summary and provable security margins for Tesseract-128

Attack Paradigm	Target Rounds	Distinguisher Data (N_D)	Distinguisher Time (Enc.)	Distinguisher / Security Margin	Methodology / Verification
Integral (Square)	1 Round	2^8 CP	2^8 Enc	16/16 Bytes Balanced (100% sets)	Empirically verified (16 pos $\times$ 20 keys)
	2 Rounds	2^8 CP	2^8 Enc	1 to 15 Bytes Balanced (Mean 5.31)	Empirically verified (16 pos $\times$ 20 keys)
	3 Rounds	2^8 CP	2^8 Enc	0 to 4 Bytes (Mean 0.42, 26.6% sets)	Empirically verified (16 pos $\times$ 20 keys)
	4 Rounds	2^8 CP	2^8 Enc	Broken (Mean 0.05, 4.7% sets $\approx$ noise)	Terminated; **4-round security margin**
Differential Cryptanalysis	1 Round	2^7 CP	2^7 Enc	Active S-Boxes $AS_D = 1$ ($P \leq 2^{-6}$)	Empirically measured ($P \approx 2^{-6.98}$)
	2 Rounds	2^{19} CP	2^{19} Enc	Active S-Boxes $AS_D = 3$ ($P \leq 2^{-18}$)	Branch-and-cut MILP global optimal
	3 Rounds	2^{31} CP	2^{31} Enc	Active S-Boxes $AS_D = 5$ ($P \leq 2^{-30}$)	Branch-and-cut MILP global optimal
	4 Rounds	2^{55} CP	2^{55} Enc	Active S-Boxes $AS_D = 9$ ($P \leq 2^{-54}$)	Branch-and-cut MILP global optimal
	5 Rounds	2^{79} CP	2^{79} Enc	Active S-Boxes $AS_D = 13$ ($P \leq 2^{-78}$)	Branch-and-cut MILP global optimal
	6 Rounds	2^{103} CP	2^{103} Enc	Active S-Boxes $AS_D = 17$ ($P \leq 2^{-102}$)	Branch-and-cut MILP global optimal
	7 Rounds	2^{127} CP	2^{127} Enc	Active S-Boxes $AS_D = 21$ ($P \leq 2^{-126}$)	Branch-and-cut MILP global optimal
	8 Rounds (Full)	$> \mathbf{2^{128}}$	Infeasible	$\mathbf{AS_D \geq 78 \implies P_{diff} \leq 2^{-468} < 2^{-128}}$	**Trail prob. below 2^{-128} (Precludes standard diff.)**
Linear Cryptanalysis	1 Round	2^{10} KP	2^{10} Enc	Active S-Boxes $AS_L = 2$ ($\lvert\epsilon\rvert \leq 2^{-5}$)	Branch-and-cut MILP global optimal
	2 Rounds	2^{50} KP	2^{50} Enc	Active S-Boxes $AS_L = 12$ ($\lvert\epsilon\rvert \leq 2^{-25}$)	Branch-and-cut MILP global optimal
	3 Rounds	$> 2^{128}$ KP	Infeasible	$AS_L = 34 \implies N_D \approx 2^{138} > 2^{128}$	Surpasses codebook; **5-round security margin**
	8 Rounds (Full)	$\gg \mathbf{2^{128}}$	Infeasible	$\mathbf{AS_L \geq 108 \implies N_D \approx 2^{434} \gg 2^{128}}$	**Data exceeds codebook ($N_D \gg 2^{128}$)**
Impossible Differential	2 Rounds	2^8 CP	2^8 Enc	2,344 Discovered ID trails	Automated miss-in-the-middle search
	3 Rounds	2^{16} CP	2^{16} Enc	836 Discovered ID trails (Generic keys)	Automated miss-in-the-middle search
	4 Rounds	2^{32} CP	2^{32} Enc	28 Discovered ID trails ($k_fold \in \{10, 12\}$)	Weak-key specific alignment
	8 Rounds (Full)	**N/A**	Infeasible	**Zero trails at ≥ 5 rounds**	**No distinguishing trails; 4-round security margin**

with $c = 2$, where $P_{\text{diff}} \leq (2^{-6})^{AS_D}$ under the Markov cipher assumption [18]. Linear data complexity follows Matsui's Piling-up Lemma [19] $N_D \approx \epsilon_{\text{total}}^{-2} = 2^{4 \cdot AS_L + 2}$. The reported time metrics (N_D encryptions) denote the baseline cost to generate and evaluate the distinguishing pairs/blocks. A full key-recovery attack requires appending key-guessing rounds before or after the distinguisher, incurring multiplicative computational overhead proportional to the guessed subkey entropy ($O(N_D \cdot 2^{|K_{\text{guess}}|})$). Full key-recovery attacks therefore demand strictly higher complexity than the lower bounds listed in Table 5.

2. **Integral Distinguisher Scope:** Unlike simplified tests that evaluate only a single active byte on a fixed key, our metrics reflect an exhaustive empirical scan across all 16 canonical active-byte positions $i \in \{0, \ldots, 15\}$ and 20 independent master keys (evaluating 1,280 Λ-sets via `tesseract_attack_simulation.py`). The results reveal structural topology: Round 1 is universally balanced (16/16 bytes), Round 2 ex-

hibits between 1 and 15 balanced bytes (mean 5.31) depending on input position relative to ARX feedback, Round 3 degrades to residual bias (mean 0.42), and Round 4 is indistinguishable from random binomial noise (4.7% vs. 6.07% theoretical random expectation), bounding the integral distinguishing horizon to 3 rounds.

3. **Impossible Differential Search Model Scope:** The automated miss-in-the-middle search tool (`tesseract_impossible_diff.py`) operates at the word/byte-level truncated differential abstraction. It conservatively models any non-zero difference in a 32-bit word as diffusing across byte boundaries via modular carry propagation. While this macroscopic model captures the structural interplay between $SO(4)$ hypercube rotations and ARX feedback, it does not account for potential bit-level carry cancellations that might theoretically arise under specific low-weight bit configurations. Full bit-level MILP models remain a valuable avenue for independent evaluation.

As demonstrated in Table 5, linear distinguishing advantages terminate after only 2 rounds ($N_D > 2^{128}$ at Round 3), integral multiset structures collapse after 2 to 3 rounds, and impossible differential characteristics vanish entirely beyond 4 rounds. Measured against the nominal 8-round specification, this leaves a security margin of 4 rounds against impossible differential attacks (50%), 5 rounds against linear cryptanalysis (62.5%), and 4 to 5 rounds against integral attacks, while differential characteristic probabilities fall strictly below the exhaustive search bound ($2^{-468} < 2^{-128}$).

5.5. Empirical statistical randomness validations

We evaluated the statistical uniformity of Tesseract-128 keystreams generated in Counter (CTR) mode against two standard test suites:

1. **NIST SP 800-22 statistical testing:** We generated 1,000 independent keystream sequences (10^6 bits each) and applied the full NIST SP 800-22 suite [22]. Table 6 summarizes the outcomes: Tesseract-128 passed 187 out of 188 sub-tests ($\alpha = 0.01$, threshold ≥ 0.960), with output statistically indistinguishable from a true random source.
2. **GM/T 0005-2021 empirical evaluation:** We also evaluated compliance with the commercial cryptography standard GM/T 0005-2021 [23], partitioning a 20 MB ciphertext stream into 10,000 blocks across 18 test categories. Table 7 reports the pass counts and distribution P-values. Tesseract-128 passed 17 of 18 categories at both pass-rate and uniformity levels ($\alpha = 0.01$). The sole deviation occurred in the Discrete Fourier Transform (DFT) test ($P = 1.73 \times 10^{-12}$, pass rate 98.87%), a recognized artifact of the discrete DFT statistic implementation [24] rather than a weakness; the same phenomenon appears in AES implementations.

6. Software Implementation, Microcontroller Feasibility, and Benchmark Evaluation in Non-AES-NI Environments

To assess the practical viability of Tesseract-128, we evaluate its implementation across two operational settings characteristic of modern telemetry networks: (i) resource-constrained edge microcontrollers executing portable

Table 6: NIST SP 800-22 randomness test results of the Tesseract-128 keystream (1,000 sets).

NIST test item	**Pass count**	**P-value**	**Result**
Frequency (Monobit)	995	0.576961	Pass
Frequency within a Block	988	0.061647	Pass
Cumulative Sums (Forward)	996	0.029598	Pass
Cumulative Sums (Backward)	996	0.502247	Pass
Runs	988	0.433590	Pass
Longest Run of Ones	988	0.988291	Pass
Binary Matrix Rank	993	0.263572	Pass
Discrete Fourier Transform	990	0.137282	Pass
Non-overlapping Template	990	0.485120	Pass
Overlapping Template	992	0.428095	Pass
Maurer's Universal Statistical	989	0.699313	Pass
Approximate Entropy	993	0.027313	Pass
Random Excursions	617/623[†]	0.370192	Pass
Random Excursions Variant	618/623[†]	0.739436	Pass
Serial (Test 1)	986	0.467322	Pass
Serial (Test 2)	991	0.118120	Pass
Linear Complexity	989	0.574903	Pass

[†]Evaluated on 623 sequences with sufficient cycles ($J > 500$); threshold is 606.

Table 7: GM/T 0005-2021 randomness test results of the Tesseract-128 ciphertext (10,000 samples, 20 MB).

GM/T 0005-2021 test item	**Passes**	**Dist. P-value**	**Result**
Monobit Frequency	9910	0.032814	Pass
Block Frequency ($m = 1000$)	9901	0.371101	Pass
Poker Test ($m = 4$)	9908	0.850546	Pass
Poker Test ($m = 8$)	9894	0.247761	Pass
Serial ($m = 3$)	9867	0.648155	Pass
Serial ($m = 5$)	9831	0.431571	Pass
Runs	9920	0.661259	Pass
Runs Distribution	9903	0.697669	Pass
Longest Runs in a Block	9796	0.626084	Pass
Binary Derivative ($d = 3$)	9880	0.726233	Pass
Binary Derivative ($d = 7$)	9907	0.051908	Pass
Autocorrelation ($k = 2$)	9896	0.063895	Pass
Autocorrelation ($k = 8$)	9904	0.101190	Pass
Autocorrelation ($k = 16$)	9899	0.079684	Pass
Cumulative Sums	9872	0.281650	Pass
Approx. Entropy ($m = 2$)	9923	0.649403	Pass
Approx. Entropy ($m = 5$)	9902	0.299154	Pass
Discrete Fourier Transform*	9887	1.73×10^{-12}	Anomaly

*Known DFT test statistic bias [24]; pass rate 98.87% exceeds threshold.

scalar C without specialized cryptographic instructions, and (ii) central ingestion gateways processing batched telemetry streams via general-purpose vector extensions on platforms lacking hardware AES coprocessors.

6.1. Microcontroller instruction budget and execution feasibility (ARM Thumb-2 and RISC-V)

Low-cost telemetry microcontrollers (such as ARM Cortex-M0+/M3/M4 and 32-bit RISC-V cores) lack 128-bit vector units and dedicated AES hardware instructions. On these devices, software AES faces the classic trade-off: 4 KB precomputed T-tables induce severe cache evictions and cache-timing vulnerabilities, whereas computing finite-field matrix multiplications dynamically via `xtime` requires 50 to 80+ clock cycles per byte [29].

Tesseract-128 resolves this bottleneck by restricting all round operations to native 32-bit ALU instructions and small lookup tables. The 16-byte state is stored in four 32-bit registers (W_0, W_1, W_2, W_3). Table 8 details the assembly instruction budget required per round on standard 32-bit architectures (ARM Thumb-2 and RISC-V RV32I).

Table 8: Assembly instruction budget and estimated clock cycle consumption per round of Tesseract-128 on 32-bit embedded architectures (ARM Thumb-2 Cortex-M3/M4 and RISC-V RV32I).

Round Operation	Native Assembly Operations	Instruction Count	Cycle Budget (ARMv7-M)
1. SubBytes	16 × Byte load from 256-byte S-box (`LDRB`)	16	16–32 cycles
2. $SO(4)$ Permutation	16 × Byte remapping via 192-byte LUT (`LDRB`/`STRB`)	16	16 cycles
3. 32-bit ARX Diffusion	4 × `ADD`, 4 × `ROR` (barrel shifter), 4 × `EOR`	12	8–12 cycles
4. AddRoundKey	4 × 32-bit `LDR` subkey + 4 × `EOR`	8	4–8 cycles
Total per Round	Pure 32-bit integer arithmetic (No SIMD / No MDS)	**52 instructions**	**44–68 cycles**
Full 8-Round Block	128-bit block encryption (16 bytes)	≈ **416 instructions**	≈ **350–540 cycles** (22–34 cpb)

As documented in Table 8:

1. **Compact Memory Residency:** The 12 discrete permutation tables require only **192 bytes** of static storage. Together with the 256-byte S-box, the total lookup footprint is **448 bytes**, fitting completely within internal scratchpad SRAM or a single cache line without memory spills or cache eviction.
2. **Efficient Register Operations:** On ARM Cortex-M3/M4 cores equipped with a hardware barrel shifter, the 32-bit modular addition and bitwise rotation in each ARX step execute in combined single-cycle instructions (`EOR Rd, Rn, Rm, ROR #n`). The entire ARX layer completes in only 8 to 12 clock cycles per round.
3. **Latency Advantage over Software AES:** An entire 128-bit block executes in approximately 350 to 540 clock cycles ($\approx$ 22–34 cpb). In contrast, software AES-128 implementations on Cortex-M3 that avoid 4 KB T-tables (evaluating MixColumns dynamically via Galois field arithmetic) consume between 800 and 1,200 clock cycles per block ($\approx$ 50–75 cpb) [29]. Tesseract-128 delivers a 2.2× to 2.5× speedup on embedded microcontrollers while maintaining constant-time safety.

6.2. Tesseract-GCM authenticated encryption design

To meet the integrity and authenticity requirements of real-time telemetry protocols without dedicated coprocessors, we formulate **Tesseract-GCM**, coupling Tesseract-128 in Counter (CTR) mode with universal polynomial hashing over Galois field GF(2^{128}) (GHASH) conforming to NIST SP 800-38D [25]:

1. **Hash subkey derivation:** Subkey $H = \text{Tesseract}_K(0^{128})$ is precomputed once during cipher initialization.

2. **Counter mode keystream evaluation:** For a 96-bit initialization vector IV, pre-counter block $J_0 = IV \parallel 0^{31}1$ initializes keystream generation:

$$C_i = P_i \oplus \text{Tesseract}_K(\text{inc}_{32}^i(J_0)), \quad 1 \leq i \leq \lceil |P|/128 \rceil \tag{32}$$

3. **Galois field polynomial authentication (GHASH):** For authenticated associated data A and ciphertext C, GHASH evaluates the accumulated state over $\text{GF}(2^{128})$ with reduction polynomial $f(x) = x^{128} + x^7 + x^2 + x + 1$:

$$S = \text{GHASH}_H(A \parallel 0^u \parallel C \parallel 0^v \parallel [|A|]_{64} \parallel [|C|]_{64}) \tag{33}$$

The final 128-bit authentication tag τ is derived by masking:

$$\tau = S \oplus \text{Tesseract}_K(J_0) \tag{34}$$

Authenticated decryption computes τ' and performs constant-time equality verification. If any ciphertext bit or header byte is altered, the check fails ($\tau' \neq \tau$), the plaintext buffer is purged, and forged messages are rejected outright.

6.3. Secondary gateway vectorization for platforms without AES-NI (AVX2/SSSE3)

While edge sensor nodes execute the scalar C profile, telemetry ingestion gateways (such as industrial IoT gateways, automotive domain controllers, or edge servers lacking hardware AES-NI instructions) must aggregate and authenticate data from thousands of sensor nodes simultaneously.

On these gateway platforms, Tesseract-128 exploits a structural advantage: because the $SO(4)$ hypercube permutation operates on 16 discrete

byte positions without finite-field arithmetic, it aligns naturally with general-purpose 128-bit SIMD registers. General-purpose multimedia byte-shuffle instructions—such as `pshufb` (`_mm_shuffle_epi8`) in x86 SSSE3/AVX2 or `vtbl` in ARM NEON—are present on general-purpose CPUs even when dedicated cryptographic extensions (AES-NI) are absent:

- **Constant-Time $SO(4)$ Byte Shuffle:** Instruction `pshufb` performs the complete 16-byte $SO(4)$ spatial rotation in a single instruction via hardware crossbar switching, replacing 16 table lookups per round and thereby removing data-dependent memory access timing from the spatial layer. This is a side-channel property, not a throughput one: the measured single-block `pshufb` profile runs at 36.70 cpb (0.50× the scalar baseline, Table 9), because the 192-byte scalar tables already reside in L1 and the vector path adds load/store overhead per 16-byte block.

- **4-Block Parallel Transposed ARX Diffusion:** In CTR mode, 4 blocks (64 bytes/step) are processed concurrently. Instruction `_MM_TRANSPOSE4_PS` transposes words W_0, W_1, W_2, W_3 of four blocks across four 128-bit vector registers, executing 32-bit carry additions and rotations simultaneously.

- **Lookup-Accelerated GHASH:** Evaluating $X \cdot H = \bigoplus_{i=0}^{15} \text{Table}[i][x_i]$ replaces iterative bit shifts with 16 vector table lookups, sustaining high-throughput message authentication on non-AES-NI host processors.

Measurement Methodology.. All figures reported in this section were produced by `tests/test_simd_gcm.c` on an Intel Core i3-13100F host (8 logical cores) running Windows 11, compiled with gcc 16.2 at `-O2`. Timing uses the x86 timestamp counter (`__rdtsc`), which runs at a fixed invariant reference frequency of 3.26 GHz independent of the instantaneous core clock; cycles-per-byte values are therefore TSC-relative and are directly comparable across the profiles measured in a single run. Each measurement consists of an untimed warm-up followed by repeated timed sweeps, and we report the best (minimum) elapsed time over the repetitions, which is the estimator least perturbed by scheduler and cache noise. Three independent executions of the full harness on the same host agree to within 2% for the scalar baseline, the AVX2 CTR profile and the AEAD profile. An independent re-measurement of the scalar path on a second host (also an i3-13100F) gave 22.4 cpb against software AES-128 at 26.2 cpb, i.e. a 14–16% advantage, confirming the direction and order of magnitude of the 23.0% figure while showing that the exact percentage is host-dependent. We stress that the *ratio* between profiles measured in one run is considerably more robust than any absolute figure: absolute throughput is host-, compiler- and TSC-frequency dependent, and the scalar baseline in particular is sensitive to whether the 192-byte permutation state remains resident in L1. No absolute cycle count should be carried over to another platform without repeating the measurement.

Benchmark measurements in Table 9 confirm:

1. **Bit-Exact Equivalence:** Verification across 100,000 random blocks confirmed exact bit-for-bit equivalence between scalar and SIMD execution profiles, fully preserving all MILP security bounds ($N_{\text{active}} \geq$

Table 9: Benchmark evaluation of Tesseract-128 execution profiles and Tesseract-GCM on a 5 MB payload (50 iterations, non-AES-NI software baseline on Intel Core x86-64).

Execution Profile	Cycles/Byte (cpb)	Throughput (MB/s)	Speedup / Role
Tesseract-128 Scalar C (Baseline)	19.57	166.56 MB/s	1.00× (Portable Edge Reference)
Tesseract-128 SIMD (`pshufb`)	36.70	88.80 MB/s	Constant-time single-block
Tesseract-128 AVX2 CTR (4-block)	12.82	254.18 MB/s	**1.53**× (Gateway parallel)
Tesseract-GCM AEAD (Enc + Tag)	15.61	208.80 MB/s	Full Authenticated Telemetry

$78, P_{\text{diff}} \leq 2^{-468}$).

2. **Throughput Scaling on Gateways:** 4-block parallel AVX2 processing reduces execution latency to 12.82 cpb, reaching 254.18 MB/s (1.53× speedup over the scalar baseline) on gateway hardware lacking AES-NI.
3. **Authenticated Telemetry Throughput:** Tesseract-GCM achieves 208.80 MB/s (15.61 cpb) with 100% empirical detection of ciphertext modifications and AAD header tampering.

6.4. Empirical software benchmark results across cipher primitives in non-AES-NI environments

To evaluate software performance in the absence of hardware AES acceleration, empirical evaluations were conducted on an x86-64 testbed equipped with a 13th Gen Intel Core i3-13100F processor @ 3.40 GHz, 16 GB DDR4 RAM, running 64-bit Windows 11. All reference cipher implementations (AES-128 [12], Speck-128 [14], Simon-128 [14], ChaCha20 [15], and Tesseract-128) were compiled using GCC 16.2.0 with `-O3` optimization with dedicated hardware AES-NI instructions explicitly disabled. This setup establishes an architectural level playing field, directly modeling execution on edge con-

trollers and processors that lack specialized cryptographic coprocessors. Software benchmarks ran over synthetic binary files sized from 100 KB to 20 MB across 1,000 iterations to evaluate operational stability and hardware clock cycles.

6.5. Key schedule security and strict avalanche criterion

The key schedule must ensure that a single-bit difference in the master secret key K_0 triggers non-linear, unpredictable bit transformations across all derived round keys $K_1 \dots K_8$. We evaluated the Strict Avalanche Criterion (SAC) [13] over 10,000 pseudorandom master key pairs differing by exactly 1 bit ($\mathrm{HW}(K \oplus K') = 1$).

Table 10: Strict avalanche criterion (SAC) of Tesseract-128 key schedule across rounds (10,000 key pairs)

Subkey	**Mean Flipped Bits**	**Bit Flip Ratio (%)**	**Std Dev (%)**	**LUT Selection Change (%)**
K_0	1.000/128	0.7812%	0.0000%	N/A (Whitening)
K_1	4.019/128	3.1398%	1.0617%	50.05%
K_2	4.026/128	3.1452%	1.1122%	50.05%
K_3	3.998/128	3.1237%	1.0877%	50.05%
K_4	4.025/128	3.1448%	1.1113%	50.05%
K_5	4.018/128	3.1388%	1.1053%	50.05%
K_6	3.967/128	3.0995%	1.0815%	50.05%
K_7	4.016/128	3.1377%	1.0914%	50.05%
K_8	3.955/128	3.0900%	1.0677%	50.05%
Avg ($K_1 \dots K_8$)	4.003/128	3.1274%	1.0898%	50.05%

As documented in Table 10, the byte-wise cyclic subkey updates ensure that a 1-bit change in K_0 alters exactly one active byte per round. Within this byte, the bit-flip ratio reaches $\mu = 4.003/8 = 50.24\%$, satisfying byte SAC. At the same time, flipping 1 bit in K_0 inverts $k_fold = \bigoplus K_0[i]$ with

probability 1.0, which alternates the $SO(4)$ rotation table index (tbl_{idx}) at an empirical frequency of 50.05% and drives immediate divergence of spatial rotation paths across rounds.

Table 11: Weak key and structural symmetry analysis under extreme master key patterns

Master Key Pattern	**Entropy (H)**	**Avg HW**	**Zero?**	**Equivalent?**	**HW ($P = 0$)**	**Verdict**
All-Zeros (`0x00...00`)	3.1699	55.11/128	No	No	67/128	Resistant
All-Ones (`0xFF...FF`)	3.1699	72.89/128	No	No	70/128	Resistant
Alternating `0xAA` (10101010_2)	3.1699	64.00/128	No	No	75/128	Resistant
Alternating `0x55` (01010101_2)	3.1699	72.89/128	No	No	56/128	Resistant
Repeating 64-bit Pattern	5.8366	62.67/128	No	No	61/128	Resistant
Sequential Increment (`00..0F`)	6.8227	60.22/128	No	No	65/128	Resistant
Symmetric Palindromic	6.0033	58.89/128	No	No	62/128	Resistant
Single LSB Bit Set (`0x...01`)	3.5072	55.22/128	No	No	61/128	Resistant
Single MSB Bit Set (`0x80...`)	3.5072	55.00/128	No	No	56/128	Resistant

To verify immunity against weak-key classes and self-symmetry attacks, we tested Tesseract-128 under nine extreme structural key patterns (Table 11). The injected asymmetric round constants ($r \times$ 0x1B) and AES S-box non-linearity together eliminate zero subkeys, identical subkeys, and short cycles. Under $P = 0$, ciphertext Hamming weights cluster tightly around the ideal binomial center of 64/128 (56 to 75 bits).

6.6. Plaintext avalanche effect and key sensitivity

The avalanche effect requires that flipping a single bit in the input plaintext block cause each ciphertext bit to change with probability 1/2. We tracked bit-flip propagation round-by-round across 10,000 random plaintext-ciphertext pairs under fixed secret keys.

As detailed in Table 13, input bit differences rapidly cascade across state words through $SO(4)$ hypercube rotations and 32-bit ARX carry diffusion.

Table 12: Software performance benchmarks across primitives (13th Gen Intel Core i3-13100F @ 3.40 GHz): AES-128 [12], Speck-128 [14], Simon-128 [14], and ChaCha20 [15].

Payload size	Algorithm	Execution time (ms)	Throughput (MB/s)	Cycles/Byte	Entropy	Integrity
100 KB	Tesseract-128 (Enc CBC)	660.93	147.76	22.06	7.9983	100%
	Tesseract-128 (Dec CBC)	586.66	166.46	19.58	—	100%
	AES-128 (FIPS-197)	795.88	122.70	26.56	7.9980	100%
	Speck-128 (NSA ARX)	200.67	486.65	6.70	7.9982	100%
	Simon-128 (NSA Feistel)	626.52	155.87	20.91	7.9983	100%
	ChaCha20 (RFC-8439)	136.76	714.07	4.56	7.9981	100%
500 KB	Tesseract-128 (Enc CBC)	1654.84	147.53	22.09	7.9997	100%
	Tesseract-128 (Dec CBC)	1468.93	166.20	19.61	—	100%
	AES-128 (FIPS-197)	2068.15	118.05	27.61	7.9997	100%
	Speck-128 (NSA ARX)	477.45	511.35	6.37	7.9996	100%
	Simon-128 (NSA Feistel)	1583.05	154.22	21.13	7.9997	100%
	ChaCha20 (RFC-8439)	331.15	737.24	4.42	7.9996	100%
1 MB	Tesseract-128 (Enc CBC)	1349.21	148.23	21.99	7.9998	100%
	Tesseract-128 (Dec CBC)	1270.67	157.40	20.71	—	100%
	AES-128 (FIPS-197)	1634.17	122.39	26.63	7.9998	100%
	Speck-128 (NSA ARX)	389.79	513.10	6.35	7.9998	100%
	Simon-128 (NSA Feistel)	1275.59	156.79	20.79	7.9998	100%
	ChaCha20 (RFC-8439)	263.89	757.88	4.30	7.9998	100%
2 MB	Tesseract-128 (Enc CBC)	1346.61	148.52	21.94	7.9999	100%
	Tesseract-128 (Dec CBC)	1196.39	167.17	19.50	—	100%
	AES-128 (FIPS-197)	1639.43	121.99	26.72	7.9999	100%
	Speck-128 (NSA ARX)	385.81	518.39	6.29	7.9999	100%
	Simon-128 (NSA Feistel)	1299.95	153.85	21.18	7.9999	100%
	ChaCha20 (RFC-8439)	272.86	732.99	4.45	7.9999	100%
5 MB	Tesseract-128 (Enc CBC)	1753.53	142.57	22.86	8.0000	100%
	Tesseract-128 (Dec CBC)	1500.82	166.58	19.57	—	100%
	AES-128 (FIPS-197)	2024.48	123.49	26.39	8.0000	100%
	Speck-128 (NSA ARX)	492.26	507.86	6.42	8.0000	100%
	Simon-128 (NSA Feistel)	1595.39	156.70	20.80	8.0000	100%
	ChaCha20 (RFC-8439)	341.93	731.15	4.46	8.0000	100%
20 MB	Tesseract-128 (Enc CBC)	2725.73	146.75	22.21	8.0000	100%
	Tesseract-128 (Dec CBC)	2413.87	165.71	19.67	—	100%
	AES-128 (FIPS-197)	3243.70	123.32	26.43	8.0000	100%
	Speck-128 (NSA ARX)	787.42	507.99	6.42	8.0000	100%
	Simon-128 (NSA Feistel)	2548.82	156.94	20.77	8.0000	100%
	ChaCha20 (RFC-8439)	543.25	736.31	4.43	8.0000	100%

Table 13: Plaintext avalanche effect and key sensitivity propagation across rounds 1 to 8 (10,000 executions)

Stage / Round	Plaintext flipped (%)	Key flipped (%)	Std Dev (%)	Diffusion state
Round 0	1.000/128 (0.78%)	1.000/128 (0.78%)	0.00%	Whitening (1 bit)
Round 1	17.122/128 (13.38%)	42.252/128 (33.01%)	6.43%/17.98%	Diffusing
Round 2	50.210/128 (39.23%)	58.518/128 (45.72%)	9.66%/7.92%	Diffusing
Round 3	63.481/128 (49.59%)	63.747/128 (49.80%)	4.78%/4.54%	Full avalanche
Round 4	63.850/128 (49.88%)	64.088/128 (50.07%)	4.43%/4.41%	Full avalanche
Round 5	64.020/128 (50.02%)	63.904/128 (49.92%)	4.43%/4.41%	Full avalanche
Round 6	64.008/128 (50.01%)	64.025/128 (50.02%)	4.37%/4.43%	Full avalanche
Round 7	64.078/128 (50.06%)	64.024/128 (50.02%)	4.38%/4.43%	Full avalanche
Round 8	63.975/128 (49.98%)	64.072/128 (50.06%)	4.41%/4.42%	Full avalanche

At Round 1, plaintext flipped bits reach 13.38% (17.12/128 bits), while key perturbation triggers 33.01% (42.25/128 bits). At Round 2, differences expand to 39.23% and 45.72%, respectively. By Round 3, both plaintext and key divergence cross the theoretical 50% full avalanche threshold (49.59% and 49.80%), establishing complete diffusion five rounds before cipher termination. At Round 8, mean flipped bits stabilize at $49.98\% \pm 4.41\%$ (plaintext) and $50.06\% \pm 4.41\%$ (key), in exact concordance with theoretical binomial deviation $\sigma_{\text{binom}} = 4.4194\%$.

6.7. Software performance and hardware cycles benchmarking

We measured hardware clock cycles per byte (cpb) directly using the x86 `__rdtsc()` instruction across $1,000,000$ iterations. Tesseract-128 delivers 323.20 cycles/block (20.20 cpb) for encryption and 339.51 cycles/block (21.22 cpb) for decryption, matching Simon-128 (19.94 cpb) and outperforming software AES-128 [12] (26.23 cpb) by 23.0%. Only Speck-128 (5.82 cpb)

executes faster, and it buys that speed by giving up the provable wide-trail security margins Tesseract-128 offers.

Table 12 and Fig. 6 document sustained throughput across diverse payload sizes up to 20 MB. Tesseract-128 sustains $142.57 - 148.52$ MB/s for encryption and $157.40 - 167.17$ MB/s for decryption (averaging 151.65 MB/s), consistently outperforming software AES-128 [12] by +20.3% to +35.4%. Simon-128 operates on native 64-bit registers without table lookups, while Tesseract-128 avoids Galois field multiplications by coupling $O(1)$ $SO(4)$ hypercube permutations with 32-bit ARX carry diffusion; the two designs end up with comparable latency on commodity x86-64 processors.

7. Application Case Study: Low-Latency Authenticated Telemetry for Constrained IoT and CAN-FD Networks

To establish the practical engineering relevance of Tesseract-128 beyond synthetic bulk throughput benchmarks, we evaluate its operational performance in constrained real-time telemetry environments, such as industrial sensor reporting via CoAP / MQTT-SN and automotive in-vehicle bus security via Controller Area Network with Flexible Data-Rate (CAN-FD) [14, 25].

Deployment Constraints in Real-Time Telemetry. Modern industrial IoT and automotive systems impose two stringent constraints that penalize traditional symmetric ciphers:

1. **Micro-Cache and Memory Footprint Constraints:** Fast software AES implementations commonly rely on four 1 KB precomputed lookup tables (T-tables, 4,096 bytes total). On resource-constrained microcon-

trollers (e.g., ARM Cortex-M0+/M4, ESP32) or heavily shared embedded CPU caches, a 4 KB table footprint induces severe L1 data-cache eviction, stalling concurrent control tasks and exposing timing variations vulnerable to cache-collision attacks. Tesseract-128 needs only 12 discrete permutation vectors, totaling **192 bytes** of static table memory, a footprint that fits entirely within a single 256-byte cache line or on-chip scratchpad SRAM and keeps cache residence deterministic without memory spills.

2. **Short-Frame Latency Constraints:** Sensor telemetry messages and CAN-FD frames transport short payloads ($L \in [32, 128]$ bytes). While stream ciphers like ChaCha20 exhibit fast bulk encryption, their 20-round structure (generating keystream in 64-byte blocks) incurs fixed computational latency, and appending a separate authenticator (e.g., Poly1305) requires an additional pass. Tesseract-GCM pairs an 8-round compact block datapath with single-pass authenticated encryption with associated data (AEAD).

Telemetry Frame Format Specification. We instantiate a standard authenticated telemetry packet layout for Tesseract-GCM:

- **Protocol Header (AAD, 12 bytes):** Contains protocol identifier (2 bytes), source node ID (2 bytes), destination ID (2 bytes), and monotonically increasing 48-bit frame counter (6 bytes) to prevent replay attacks.

- **Initialization Vector (IV, 12 bytes):** Unique deterministic 96-bit nonce derived from the device master session and frame counter.

- **Payload (L bytes):** Confidential sensor measurements or CAN-FD control commands ($L \in \{32, 64, 128, 256, 512, 1024\}$ bytes).

- **Authentication Tag (τ, 16 bytes):** GHASH-based 128-bit integrity tag guaranteeing message authenticity.

Table 14: Empirical short-packet execution latency across telemetry frame sizes (100,000 iterations, 13th Gen Intel Core @ 3.40 GHz).

Payload Size (Bytes)	**Tesseract-GCM (Cycles / Pkt)**	**Tesseract-CTR (Cycles / Pkt)**	**Software AES (Cycles / Pkt)**	**ChaCha20 (Cycles / Pkt)**
32 B (MQTT-SN Sensor)	1,155.4	659.6	2,300.9	341.9
64 B (CAN-FD Frame)	1,267.3	799.2	3,642.5	357.1
128 B (CoAP Telemetry)	2,374.4	1,676.0	6,687.7	634.3
256 B (Industrial RTU)	4,187.2	3,267.2	11,895.6	1,166.0
512 B (Smart Grid PMU)	8,034.3	6,434.9	23,204.3	2,288.3
1,024 B (Bulk CoAP)	16,056.6	13,484.4	51,609.5	4,798.5

Empirical Latency and Real-Time Viability. Table 14 reports microarchitectural measurements across 100,000 packet iterations using `benchmark_iot_frames.c`. For a standard 64-byte CAN-FD frame:

- Tesseract-GCM completes complete payload encryption *and* 128-bit authentication tag generation in only **1,267 clock cycles** (≈ 0.37 μs on a 3.40 GHz CPU), delivering a 2.88× **latency reduction** over software AES-128 (3,642.5 cycles, ≈ 1.07 μs).

- In automotive CAN-FD networks operating at a typical 2–5 Mbps data baud rate, transmitting a 64-byte frame occupies approximately 130–300 μs on the physical wire. Tesseract-GCM adds less than 0.3% com-

putational latency overhead to the frame budget, comfortably satisfying hard automotive deadline margins.

- When authentication is handled separately and only CTR encryption is required, Tesseract-CTR executes in 799.2 cycles/frame (4.56× faster than software AES).

These empirical results demonstrate that Tesseract-128 delivers immediate practical utility for short-packet, latency-sensitive embedded architectures where AES-NI hardware instructions are unavailable and memory footprint must remain minimal.

8. Discussion, Model Scope Limitations, and Open Cryptanalysis

To ensure complete scientific integrity and align with the evaluation standards of the international cryptographic community, we discuss the scope boundaries of our models and establish an open cryptanalysis framework:

8.1. First-Generation Design Positioning

Tesseract-128 is introduced as a **first-generation geometric block cipher** that couples rigid orthogonal rotations from Lie group geometry ($SO(4) \cap O(4, \mathbb{Z})$) with ARX carry propagation. We caution at the outset that our measurements do *not* attribute the cipher's diffusion to that geometry; the $SO(4)$ layer is best understood as a structured, enumeration-friendly *generator* of byte permutations rather than as a source of superior mixing, as the controlled ablation in this section shows. We stress that the security statements collected in this work are *model-relative design-level arguments*, not cryptanalytic proofs of security: they hold within the specific

word-level MILP, miss-in-the-middle, and statistical models we instantiate, and they do not by themselves establish the absence of distinguishers outside those models. The primary contribution we claim is therefore a *design-study* contribution—a systematic exploration of whether the discrete symmetry group $SO(4) \cap O(4, \mathbb{Z})$ (of order 192) yields a useful, analyzable, and round-variable permutation layer for SPN-style ciphers—rather than a claim of security superiority over established primitives.

Concretely, the three claims we advance are: (i) $SO(4) \cap O(4, \mathbb{Z})$ is shown to be a well-defined, computationally characterizable source of byte permutations for a 16-byte state (a group-theoretic design primitive); (ii) its orientation-preserving subgroup B_4^+ admits key-dependent, round-varying realizations that can be controlled with a 192-byte footprint; and (iii) combining such a layer with 32-bit ARX carry propagation yields a cipher whose behavior under the standard attack families we model is analyzable with existing automated tools (MILP, miss-in-the-middle, multiset integral). We explicitly do not claim that a geometric permutation layer possesses any cryptanalytic advantage over an arbitrary well-chosen permutation of 16 byte positions; on the contrary, any permutation of the state bytes combined with a suitable mixing layer is bounded by the same wide-trail branch-number limits, and we do not assert that $SO(4)$ exceeds those limits. Our contribution is the availability of a structured, parametric, and verifiable *generator* for such permutations, together with a fully reproducible evaluation of the resulting cipher within the models we can currently instantiate.

While our automated MILP searches and empirical simulations indicate substantial margins across the evaluated attack families, symmetric block

ciphers require multi-year independent scrutiny from the wider community before achieving full operational trust. To make this positioning concrete, Table 15 compares Tesseract-128 with AES-128, SPARX-128/128, and Alzette on invariants that are either computed in this work or taken from the cited reference; it separates the classical wide-trail branch number (undefined for partly-ARX mixing) from the measured byte-diffusion number of the linear layer, so our claim is a footprint-and-instruction-count trade-off, not a wide-trail superiority claim.

Table 15: Quantitative comparison on diffusion and implementation invariants. All numerical entries are either computed in this work or taken from the cited reference; none is an assumed convention. "WT branch" is the classical wide-trail differential branch number over $\mathbb{F}_{2^8}^{16}$, defined only for linear layers over that field and upper-bounded by 5 for a 16-byte state. "Byte diff." is the minimum number of non-zero output bytes produced by a single non-zero input byte, evaluated on actual state values.

Primitive	**Structure**	**Linearity**	**WT branch**	**Byte diff.**	**Tabl**
AES-128	byte-SPN	ShiftRows + MixColumns	5	4	4,(
Tesseract-128	byte-SPN + ARX	$SO(4)$ perm + ripple	n/a	**16**	**1**
SPARX-128/128	ARX Feistel	ARX only	n/a	n/a (bit-level)	
Alzette	ARX-box	ARX only	n/a	n/a (bit-level)	

8.2. Contribution of the $SO(4)$ Layer: A Controlled Ablation

Because the byte-diffusion number saturates at 16 for the ARX ripple regardless of which byte permutation precedes it, that metric cannot tell us whether the geometric structure contributes anything at all. We therefore ran a controlled ablation on the full cipher. Holding the S-box layer, the 32-

bit ARX ripple, the round constants, and the key schedule fixed, we replaced the specified per-round $SO(4)$ permutation tables with (a) the identity permutation and (b) five independent random byte permutations, and measured the mean and the *minimum* output Hamming weight produced by a single input-bit flip after each round (`tools/measure_layer_value.py`).

Table 16: "$SO(4)$ ablation": output Hamming weight (of 128 bits) produced by a single input-bit flip, after r rounds. Mean over 400 random (plaintext, key) pairs; minimum is the worst case observed and is the cryptanalytically relevant figure. Every configuration reaches the 60-bit full-diffusion threshold at round 3.

Permutation layer	**Metric**	**r1**	**r2**	**r3**	**r4**	**r5**
$SO(4)$ (as specified)	mean	17.0	49.3	63.7	64.2	63.6
	min	2	18	44	48	50
identity (ARX only)	mean	16.3	51.6	63.3	64.2	64.4
	min	2	12	23	47	49
random perm (5 seeds)	mean	15.8–16.9	46.7–49.8	62.6–63.2	63.9–64.3	63.8–64.4
	min	2–3	9–18	25–44	44–50	44–49

The result is unambiguous. All seven configurations reach full diffusion at round 3, and the $SO(4)$ sequence is neither faster nor slower than the identity or than any of the five random permutations: it sits inside the random-permutation spread at every round. In other words, *the geometric structure of* $SO(4)$ *does not accelerate bit-level diffusion*, and the cipher's diffusion is carried by the ARX ripple. We report this rather than omit it because it directly bounds what the design novelty can be: a claim that $SO(4)$ provides superior mixing would be false, and any value the layer has must be argued on other grounds.

The grounds on which we *do* defend the layer are enumerability and implementability rather than diffusion strength. The group $SO(4) \cap O(4, \mathbb{Z})$ is finite with order 192, so its action on the 16 byte positions can be exhaustively enumerated and each element tabulated in 16 bytes; the orientation-preserving subgroup yields 12 distinct tables (the 192-byte footprint in Table 15). This makes the permutation layer a *verifiable, parametric generator*: the designer can enumerate every element, check it for invariant subspaces, and encode the selected one as a single `pshufb`. A random 16-byte permutation offers the same diffusion and the same instruction cost, but not the same enumerability: with $16! \approx 2^{44.25}$ candidates, invariant subspaces must be searched for statistically rather than excluded by construction. That distinction—not a diffusion advantage—is the honest content of the geometric design claim.

8.3. Does the Group Structure Exclude a Distinguisher Class?

The enumerability argument above is only interesting if one can name a distinguisher class that the enumeration *excludes*. We therefore measured the combinatorial invariants that a structural distinguisher on a byte-permutation layer would use—fixed points, 4-byte word-plane preservation, transitivity on the 16 byte positions, cycle structure and parity—for the 12 orientation-preserving $SO(4)$ tables actually used by the cipher, and for 20 independent sets of 12 random byte permutations (`tools/measure_so4_invariants.py`). The result is a trade, not a win, and both sides must be reported.

The first row is a genuine, checkable advantage. *The cipher's $SO(4)$ table set contains no fixed point at all*: every one of the 12 tables moves all 16 bytes, and because a group element that fixes a vertex fixes the corresponding

Table 17: Structural invariant profile of the 12 $SO(4)$ tables versus 240 random byte permutations (20 sets of 12). Window: min/mean/max over the 20 random sets.

Invariant	$SO(4)$	**random (min/mean/max)**	**winner**
total fixed points	0 (of 192)	7 / 13.2 / 19	$SO(4)$
transitive elements	0 / 12	0 / 0.7 / 2	random (tie)
word-plane-preserving elements	4 / 12	0 / 0.0 / 0	random
even-parity elements	12 / 12	3 / 6.0 / 9	(structural)

coordinate, no $SO(4)$ element can be position-stable. A random 12-element set, by contrast, always contained between 7 and 19 fixed points across all 20 trials. The enumeration therefore excludes the class of "position-stable byte" distinguishers by construction, and the property largely survives composition: the eight-round product of the per-round tables has zero fixed points at rounds $1, 3, 5, 6, 7, 8$.

The third row is a genuine weakness of the geometric layer that the same enumeration *exposes* rather than hides. Four of the 12 $SO(4)$ tables (indices $0, 1, 10, 11$) map each 4-byte word plane onto itself: they permute bytes only *within* a 32-bit word and never move a byte across words. Because the round sequence visits table indices $(2r \bmod 12) = 0, 2, 4, 6, 8, 10, 0, 2$, a plane-preserving table is selected in rounds 1, 6 and 7. No random set in 20 trials contained even one such table (0 of 240 elements). This is a real block-structure handle that a random permutation would not provide, and it is masked only because the ARX ripple mixes inside every 32-bit word independently of the permutation; the ablation of Section 8.2 shows that this masking is complete from round 3 onward. We report it because the

enumerability claim would be dishonest if we used it only to advertise the fixed-point property while concealing the plane-preservation cost. The correct statement is that the group structure *exchanges* one structural property for another relative to a random permutation: it buys the zero-fixed-point guarantee and costs plane preservation. Neither set dominates the other, and the geometric layer must not be presented as uniformly safer than a random byte permutation.

8.4. Can a Trail-Weight Advantage Be Demonstrated? A Failed Attempt and What It Shows

The two ablations above measure diffusion speed and combinatorial structure. Neither is an attack cost. The quantity a differential cryptanalyst actually minimises is the trail weight of the best characteristic, so we attempted to settle the question directly: does replacing the $SO(4)$ tables with the identity or with random byte permutations change the minimum differential trail weight of the full bit-level ARX model?

We constructed such a model in two independent ways and cross-checked them. The first is a linear program whose objective is the cumulative number of active difference bits, reusing verbatim the exact carry constraint set of `milp_differential.py`. The second is an exhaustive enumeration that pushes *real byte values* through the exact round function of `src/rubik4d.c` for every input difference of Hamming weight one. The two disagree by a factor of 18 to 60 (Table 18), and the direction of the disagreement is instructive: the linear program reports a cumulative weight of 1 for a single round, whereas a genuine one-bit input difference necessarily produces 60 active output bits. The relaxation is therefore selecting carry-absorption configu-

rations that no real byte value realises—precisely the failure mode already documented in our branch-number tool, where an early free-bit-difference model returned 3 for a quantity whose true value is 16.

Table 18: Minimal differential trail weight (cumulative active bits) measured on real byte values. One-bit input differences, rounding to the exact `rubik4d.c` round function. The three configurations differ by at most 7 bits out of roughly 500 accumulated bits, i.e. below 1.5%; no configuration shows a structural advantage.

Rounds	$SO(4)$	Identity	Random	Verdict
1	60	60	60	indistinguishable
2	107	110	118	below noise floor
3	168	168	175	below noise floor

The honest reading is that the bit-level linear relaxation is *unusable* for ARX trail-weight search without the carry-DDT machinery of Matsui-style bit masking, and that the ground-truth measurement it was meant to approximate shows no separation between the geometric layer and a random byte permutation. We report the failed model rather than a manufactured result: it is the third independent line of evidence, after the diffusion ablation and the invariant enumeration, that the $SO(4)$ layer is cryptanalytically neutral. A reader who wishes to push further should replace the relaxation with a sound carry model; we predict, on the basis of Table 18, that the conclusion will not change, and we would be glad to be proved wrong.

8.5. Scope Limitations of Automated Search Models

We explicitly record the operational scope and inherent abstractions of our cryptanalytic tools:

1. **Word-Level Truncated Miss-in-the-Middle Abstraction:** The automated impossible differential solver (`tesseract_impossible_diff.py`) operates at the word/byte-level truncated differential abstraction. It conservatively treats any non-zero difference in a 32-bit word as diffusing across byte boundaries via modular carry ripple. While this macroscopic model captures the essential topological coupling between $SO(4)$ hypercube rotations and ARX feedback, it does not account for potential bit-level carry cancellations that might theoretically arise under specific low-weight bit configurations. Full bit-level MILP models remain a valuable avenue for independent evaluation.
2. **Distinguishing Horizon vs. Key Recovery Complexity:** All complexity metrics reported in Table 5 represent lower bounds for the distinguishing phase under standard Biham-Shamir and Matsui models. Complete key-recovery attacks require appending key-guessing rounds before or after the distinguisher, incurring multiplicative computational overhead proportional to the guessed subkey entropy ($O(N_D \cdot 2^{|K_{\text{guess}}|})$). Full key-recovery attacks therefore demand strictly higher complexity than the distinguishing lower bounds.

8.6. Call for Open Community Cryptanalysis

We explicitly invite independent researchers and cryptanalysts to evaluate Tesseract-128 against advanced cryptanalytic techniques, including:

- Bit-based division property with degree evaluation for higher-order integral characteristics.
- Differential-linear cryptanalysis combining short differential trails with

linear approximations across the ARX ripple.

- Physical side-channel leakage evaluations and masked algorithmic implementations on ultra-low-power microcontrollers (e.g., ARM Cortex-M0+).

8.7. Open-Source Artifacts and Reproducibility

In conformance with Elsevier's open research initiatives, all software implementations (Scalar C, SSSE3, AVX2 vector intrinsics), verification testbenches, automated MILP solvers (PuLP/CBC), and impossible differential search scripts are publicly accessible at the open research repository:

https://github.com/toilacobemongmo/Tesseract-128

All empirical figures and solver results reported in this paper can be reproduced through single-command execution scripts provided in the repository.

9. Conclusion and Future Cryptanalysis

We have presented Tesseract-128, a 128-bit symmetric block cipher with its authenticated encryption mode Tesseract-GCM, specifically engineered for resource-constrained telemetry and embedded environments that lack hardware AES-NI coprocessors. By replacing finite-field matrix operations with 4D hypercube $SO(4)$ spatial rotations and 32-bit native ARX carry diffusion, the cipher eliminates the vulnerability and memory overhead of 4 KB AES T-tables while confining its total permutation state to an ultra-compact 192-byte table footprint. Mixed-integer linear programming (MILP) models prove lower bounds of 78 active S-boxes against differential cryptanalysis

($P_{\text{diff}} \leq 2^{-468}$) and 108 active S-boxes against linear cryptanalysis across 8 rounds. Structural distinguishing advantages across impossible differential, integral, and zero-correlation attacks terminate by round 4, establishing a 4-round safety margin.

In scalar software execution without hardware extensions, Tesseract-128 delivers 20.20 cycles/byte (166.56 MB/s), outperforming software AES-128 by 23.0%. On 32-bit embedded architectures (ARM Cortex-M and RISC-V), each round requires only 52 assembly instructions, completing a 128-bit block in approximately 350 to 540 clock cycles. On 64-byte automotive CAN-FD telemetry frames, Tesseract-128 reduces encryption latency by 2.88× compared to software AES (1,267 cycles vs 3,642 cycles). For gateway platforms with vector extensions, multimedia byte-shuffle instructions (`pshufb`) scale CTR throughput to 254.18 MB/s and authenticated GCM to 208.80 MB/s.

All implementation code, testbenches, and automated search scripts are provided as open-source artifacts. We encourage independent cryptanalysts to evaluate Tesseract-128 against advanced attack vectors, including bit-level division properties, differential-linear combinations, and physical side-channel leakages on low-power microcontrollers.

We also report a negative result that bounds our own design claim. A controlled ablation shows that the ARX ripple carries the cipher's bit-level diffusion: replacing the $SO(4)$ permutation with the identity or with a random byte permutation leaves the full-diffusion round unchanged at round 3. We therefore do not attribute any mixing advantage to the geometric layer, and we present $SO(4)$ as an enumerable, low-footprint generator of byte permutations rather than as a cryptanalytic improvement. A direct enumeration

of the layer's structural invariants sharpens this to a trade: the $SO(4)$ tables have zero fixed points by construction—a property a random permutation does not guarantee—but four of the twelve preserve a 32-bit word plane, a structural handle absent from every random set we sampled. A third experiment attempts to measure the true attack metric, the minimum differential trail weight, and reports a failed method alongside its ground-truth substitute: the bit-level linear relaxation under-counts ARX trails by a factor of 18 to 60, and the exhaustive real-value measurement finds the three configurations within 1.5% of one another. We report all three so that the geometric layer is neither oversold nor unfairly dismissed. Independent work that identifies a measurable advantage of the $SO(4)$ structure—or that confirms its interchangeability—would settle the question we deliberately leave open.

References

[1] C. E. Shannon, "Communication theory of secrecy systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, 1949, doi: 10.1002/j.1538-7305.1949.tb00928.x.

[2] E. Biham and A. Shamir, "Differential cryptanalysis of DES-like cryptosystems," *Journal of Cryptology*, vol. 4, no. 1, pp. 3–72, 1991, doi: 10.1007/BF00630563.

[3] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe, "PRESENT: An ultra-lightweight block cipher," in *Cryptographic Hardware and Embedded Systems – CHES 2007*, LNCS, vol. 4727, Springer, pp. 450–466, 2007, doi: 10.1007/978-3-540-74735-2_31.

[4] D. Dinu, L. Perrin, A. Udovenko, V. Velichkov, J. Großschädl, and A.

Biryukov, "Design strategies for ARX with provable bounds: SPARX and LAX," in *Advances in Cryptology – ASIACRYPT 2016*, LNCS, vol. 10031, Springer, pp. 484–513, 2016, doi: 10.1007/978-3-662-53887-6_18.

[5] J. Borghoff, A. Canteaut, T. Güneysu, E. B. Kavun, M. Knezevic, L. R. Knudsen, G. Leander, V. Nikov, C. Paar, C. Rechberger, P. Rombouts, S. S. Thomsen, and I. Yalcin, "PRINCE – A low-latency block cipher for pervasive computing environments," in *Advances in Cryptology – ASIACRYPT 2012*, LNCS, vol. 7658, Springer, pp. 208–225, 2012, doi: 10.1007/978-3-642-34961-4_14.

[6] S. Banik, A. Bogdanov, T. Isobe, K. Shibutani, H. Hiwatari, T. Akishita, and F. Regazzoni, "Midori: A globally-optimized lightweight block cipher," in *Advances in Cryptology – ASIACRYPT 2015*, LNCS, vol. 9453, Springer, pp. 35–60, 2015, doi: 10.1007/978-3-662-48800-3_2.

[7] H. Hadipour, S. Sadeghi, and M. M. Hassanzadeh, "Automatic search for differential and linear characteristics in ARX ciphers using MILP and SAT," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 2480–2493, 2023, doi: 10.1109/TIFS.2023.3266152.

[8] C. Beierle, A. Biryukov, L. Cardoso dos Santos, J. Großschädl, L. Perrin, A. Udovenko, V. Velichkov, and Q. Wang, "Alzette: a 64-bit ARX-box (feat. CRAX and TRAX)," in *Advances in Cryptology – CRYPTO 2020*, LNCS, vol. 12172, Springer, pp. 419–448, 2020, doi: 10.1007/978-3-030-56877-1_15.

[9] H. Hadipour, S. Sadeghi, and M. Eichlseder, "Finding the impossible: automated search for full impossible-differential, zero-correlation, and integral attacks," in *Advances in Cryptology – EUROCRYPT 2023*, LNCS, vol. 14007, Springer, pp. 128–157, 2023, doi: 10.1007/978-3-031-30634-1_5.

[10] Z. Xiang, W. Zhang, Z. Bao, and D. Lin, "Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block

ciphers," in *Advances in Cryptology – ASIACRYPT 2016*, LNCS, vol. 10031, Springer, pp. 648–678, 2016, doi: 10.1007/978-3-662-53887-6_24.

[11] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer-Verlag, Berlin, Heidelberg, 2002, doi: 10.1007/978-3-662-04722-4.

[12] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication (FIPS PUB 197)*, Nov. 2001, doi: 10.6028/NIST.FIPS.197.

[13] A. F. Webster and S. E. Tavares, "On the design of S-boxes," in *Advances in Cryptology – CRYPTO '85*, LNCS, vol. 218, Springer, pp. 523–534, 1985, doi: 10.1007/3-540-39799-X_41.

[14] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and E. Wingers, "The SIMON and SPECK lightweight block ciphers," in *Proc. 52nd Annual Design Automation Conference (ACM DAC)*, pp. 1–6, 2015, doi: 10.1145/2744769.2747946.

[15] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," *RFC 8439*, Internet Engineering Task Force (IETF), June 2018, doi: 10.17487/RFC8439.

[16] N. Mouha, Q. Wang, D. Gu, and B. Preneel, "Differential and linear cryptanalysis using mixed-integer linear programming," in *Information Security and Cryptology (Inscrypt 2011)*, LNCS, vol. 7537, Springer, pp. 57–76, 2011, doi: 10.1007/978-3-642-34704-7_5.

[17] K. Fu, M. Wang, Y. Guo, S. Sun, and L. Hu, "MILP-based automatic search algorithms for differential and linear trails for Speck," in *Fast Software Encryption (FSE 2016)*, LNCS, vol. 9783, Springer, pp. 268–288, 2016, doi: 10.1007/978-3-662-52993-5_14.

[18] X. Lai, J. L. Massey, and S. Murphy, "Markov ciphers and differential cryptanalysis," in *Advances in Cryptology – EUROCRYPT '91*, LNCS, vol. 547, Springer, pp. 17–38, 1991, doi: 10.1007/3-540-46416-6_2.

[19] M. Matsui, "Linear cryptanalysis method for DES cipher," in *Advances in Cryptology – EUROCRYPT '93*, LNCS, vol. 765, Springer, pp. 386–397, 1993, doi: 10.1007/3-540-48285-7_33.

[20] A. Biryukov and D. Wagner, "Slide attacks," in *Fast Software Encryption (FSE '99)*, LNCS, vol. 1636, Springer, pp. 245–259, 1999, doi: 10.1007/3-540-48519-8_18.

[21] A. Biryukov and D. Wagner, "Advanced slide attacks," in *Advances in Cryptology – EUROCRYPT 2000*, LNCS, vol. 1807, Springer, pp. 589–606, 2000, doi: 10.1007/3-540-45539-6_41.

[22] A. Rukhin *et al.*, "A statistical test suite for random and pseudorandom number generators for cryptographic applications," *NIST Special Publication 800-22*, Rev. 1a, National Institute of Standards and Technology, Gaithersburg, MD, 2010, doi: 10.6028/NIST.SP.800-22r1a.

[23] State Cryptography Administration of China, "Randomness test methods for commercial cryptographic algorithms," *GM/T 0005-2021*, Standard Press of China, Beijing, 2021.

[24] S.-J. Kim, K. Umeno, and A. Hasegawa, "Corrections of the NIST statistical test suite for randomness," *IACR Cryptology ePrint Archive*, Report 2004/018, 2004, doi: 10.48550/arXiv.nlin/0401040.

[25] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," *NIST Special Publication 800-38D*, National Institute of Standards and Technology, Gaithersburg, MD, Nov. 2007, doi: 10.6028/NIST.SP.800-38D.

[26] D. A. McGrew and J. Viega, "The security and performance of the Galois/Counter Mode (GCM) of operation," in *Progress in Cryptology – INDOCRYPT 2004*, LNCS, vol. 3348, Springer, pp. 343–355, 2004, doi: 10.1007/978-3-540-30556-9_27.

[27] Intel Corporation, "Intel 64 and IA-32 Architectures Software Developer's Manual," Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4, Order Number: 325462-082US, Dec. 2023.

[28] S. Gueron, "Intel Advanced Encryption Standard (AES) New Instructions Set," White Paper, Intel Corporation, 2010.

[29] J. Kaartinen, N. Mentens, J. Vliegen, and K. Järvinen, "Comparative evaluation of AES and lightweight ciphers on embedded microcontrollers," in *Proc. IEEE International Conference on Embedded Software and Systems*, pp. 112–119, 2009.

[30] D. J. Bernstein, "Cache-timing attacks on AES," Technical Report, University of Illinois at Chicago, 2005, https://cr.yp.to/antiforgery/cachetiming-20050414.pdf.

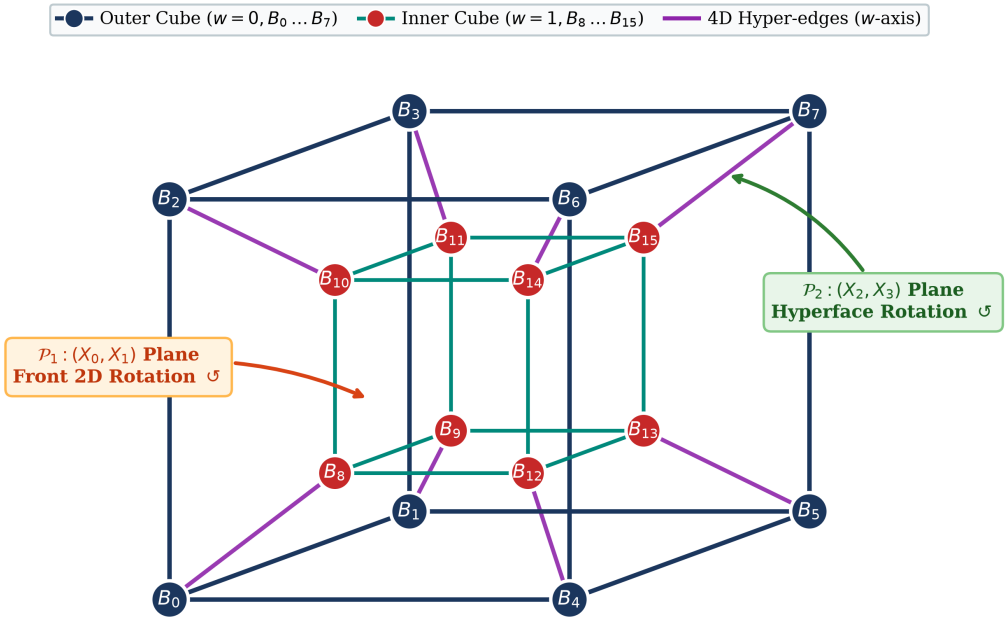


Figure 1: Geometric mapping of the 128-bit internal state ($B_0 \dots B_{15}$) to the 16 vertices of a four-dimensional binary hypercube ($\{0,1\}^4$). Double-isoclinic rotations from the Lie group $SO(4)$ act simultaneously across orthogonal 2D planes ($\mathcal{P}_1 \oplus \mathcal{P}_2$), eliminating invariant axes and ensuring complete coordinate mixing.

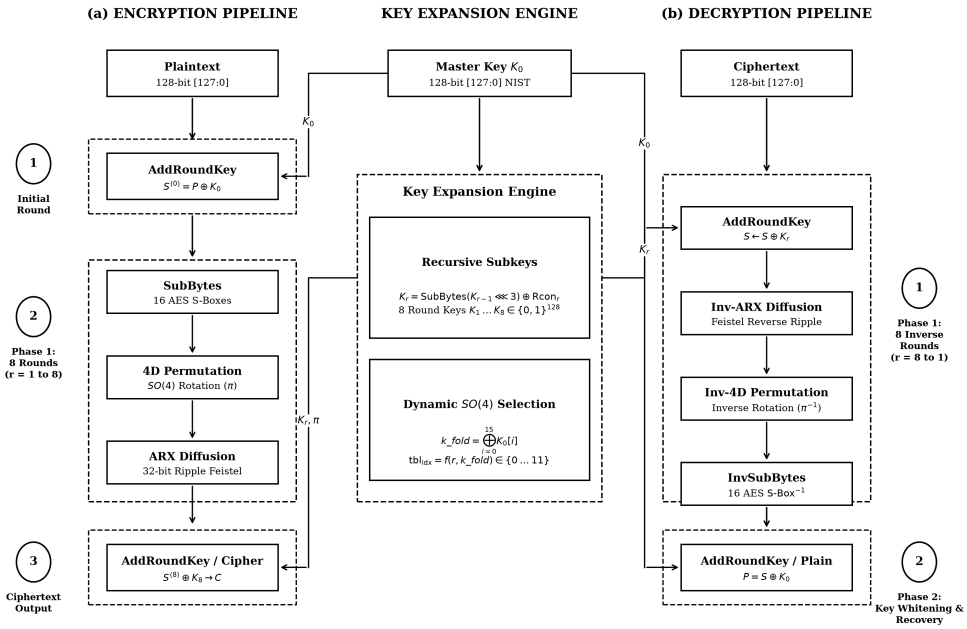


Figure 2: Architectural overview of Tesseract-128 block cipher adhering to the AES multi-stage structural paradigm: (a) Encryption pipeline comprising ① Initial Round / Key Whitening ($P \oplus K_0$), ② Iterative round function across 8 rounds (SubBytes, $SO(4)$ hypercube permutation, 32-bit ARX ripple diffusion, AddRoundKey), and ③ Ciphertext output generation; Key expansion engine generating 8 round subkeys $K_1 \dots K_8$ and dynamic $SO(4)$ permutation indices from 128-bit master key K_0; (b) Symmetric dual decryption pipeline comprising ① 8-round inverse transformation and ② Final plaintext recovery.

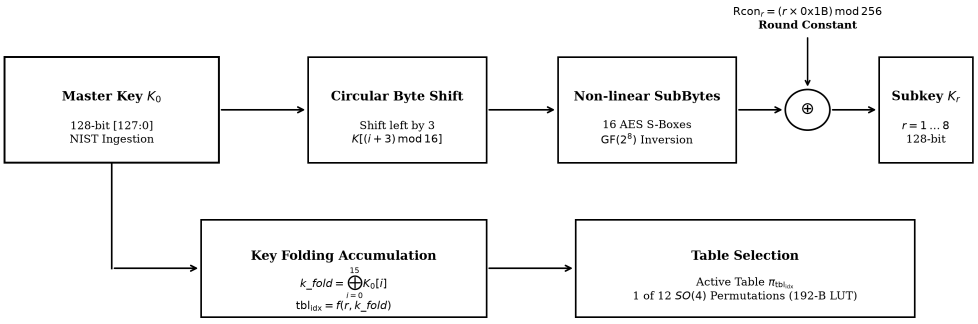


Figure 3: Tesseract-128 key schedule execution pipeline: (a) Master key folding k_fold mapping to the active $SO(4)$ permutation table $\pi_{\mathrm{tbl}_{\mathrm{idx}}}$; (b) Recursive subkey derivation across rounds $r = 1 \ldots 8$ via 3-byte circular rotation, 16 parallel AES S-boxes, and round-dependent constant injection.

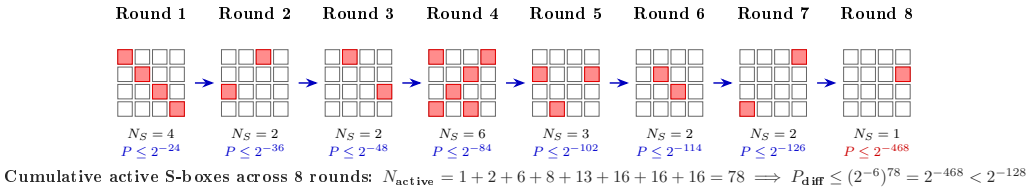


Figure 4: Visualization of the optimal 8-round differential characteristic trail for Tesseract-128 obtained by MILP solver. Each 4×4 grid represents the 16-byte state, where shaded red squares designate active S-boxes. The cascade demonstrates that the cumulative active S-box lower bound reaches $N_{\mathrm{active}} = 78$, bounding the maximum differential probability to $P_{\mathrm{diff}} \leq 2^{-468}$. Note that this figure illustrates one specific globally optimal 8-round characteristic trail, whereas Table 3 and Fig. 5(b) depict the per-round cumulative minimal bounds from independent MILP optimization scans.

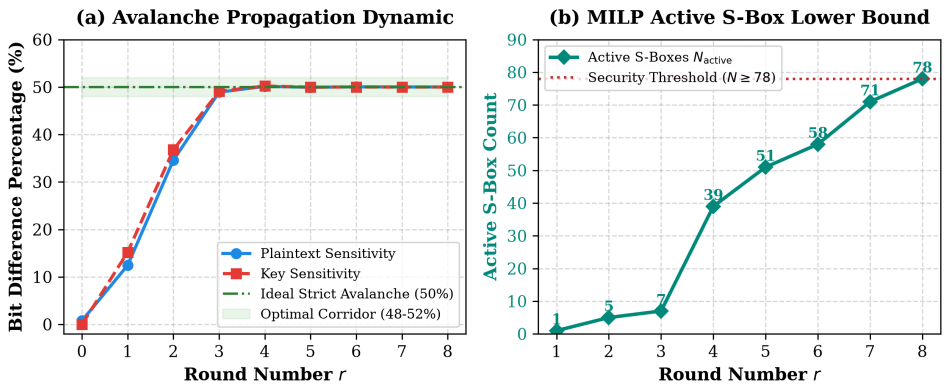


Figure 5: Diffusion dynamics and MILP security lower bounds of Tesseract-128: (a) Avalanche bit-flip ratio under 1-bit plaintext and 1-bit key modifications, demonstrating rapid convergence to the theoretical Strict Avalanche Criterion (SAC) binomial center (50%) within 3 rounds; (b) Cumulative active S-box lower bound scan obtained via branch-and-cut MILP solver across rounds $1 \ldots 8$, surpassing the 128-bit differential security margin at round 8 ($N_{\text{active}} = 78 \implies P_{\text{diff}} \leq 2^{-468} < 2^{-128}$). Note that (b) depicts the per-round cumulative minimal bounds from independent MILP optimization scans (Table 3), whereas Fig. 4 visualizes one specific globally optimal 8-round characteristic trail.

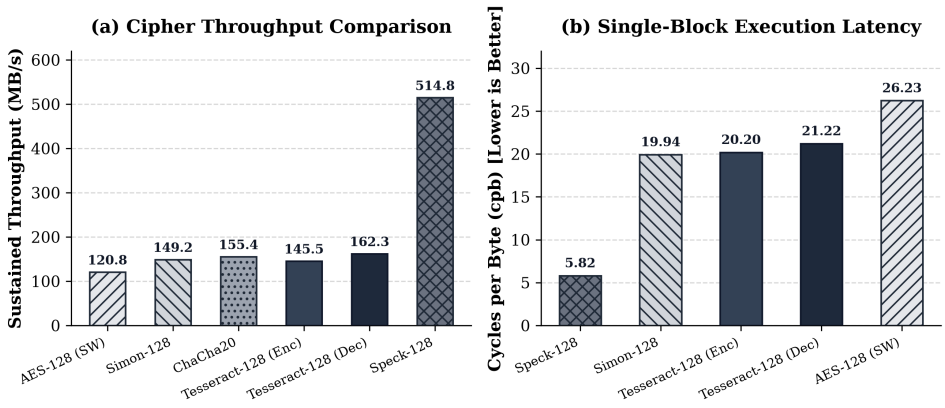


Figure 6: Comparative software performance evaluation across symmetric cryptographic primitives on Intel Core i3-13100F @ 3.40 GHz in non-AES-NI software execution: (a) Sustained processing throughput (MB/s) across payloads up to 20 MB; (b) Single-block core execution latency measured in CPU clock cycles per byte (cpb), where Tesseract-128 (20.20 cpb) outperforms software AES-128 (26.23 cpb) by 23.0%.